

Object Detection

Part – 1

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

Email: oybekeraliev7@gmail.com



Object Detection

What is Object Detection?

- Object detection, within computer vision, involves identifying objects within images or videos.
- These algorithms commonly rely on machine learning or deep learning methods to generate valuable outcomes.

Object Detection

What is Object Detection?

- So instead of classifying, which type of dog is present in these images, we have to actually locate a dog in the image.
- That is, I have to find out where is the dog present in the image?



Object Detection

What is Object Detection?

- Now the next question comes into the human mind, how can we do that?
- We can create a box around the dog that is present in the image and specify the x and y coordinates of this box.



Object Detection

What is Object Detection?

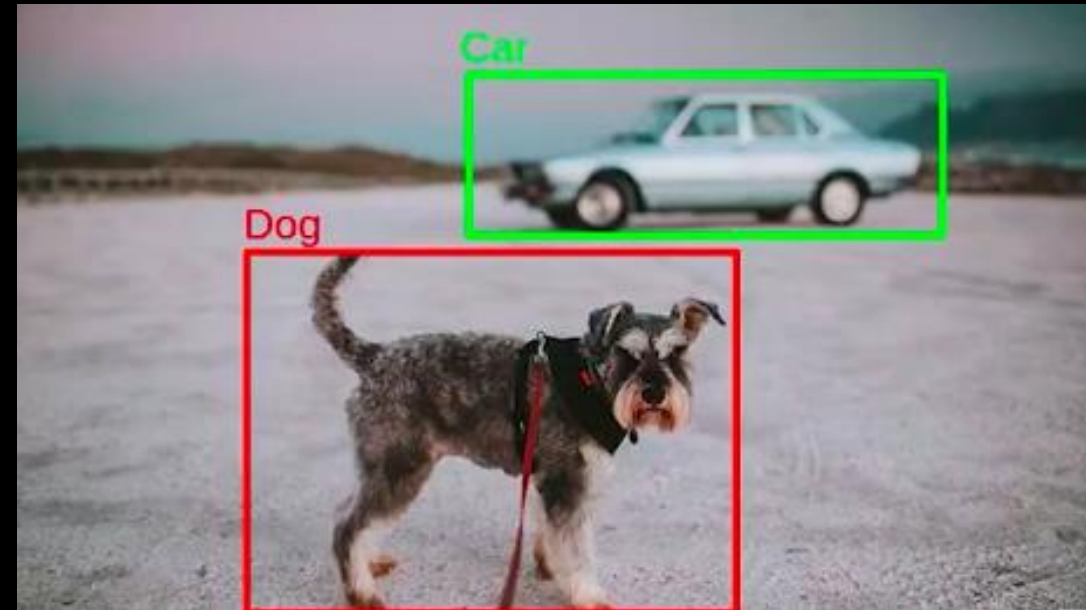
- For now, consider that the location of the object in the image can be represented as coordinates of these boxes.
- This box around the object in the image is formally known as a **bounding box**. Now, this becomes an **image localization** problem where we are given a set of images and we have to identify where is the object present in the image.



Object Detection

What is Object Detection?

- Note that here we have a single class.
what if we have multiple classes?
- In this image, we have to locate the objects in the image but note that all the objects are not dogs.
- Here we have a dog and a car. So we not only have to locate the objects in the image but also **classify** the located object as a dog or Car.
- So this becomes an **object detection** problem.



Object Detection

What is Object Detection?

Image Classification



- Object Classification

V/S

Object Detection



- Object Classification
- Object Localization

Object Detection

Object Localization

What are localization and detection?

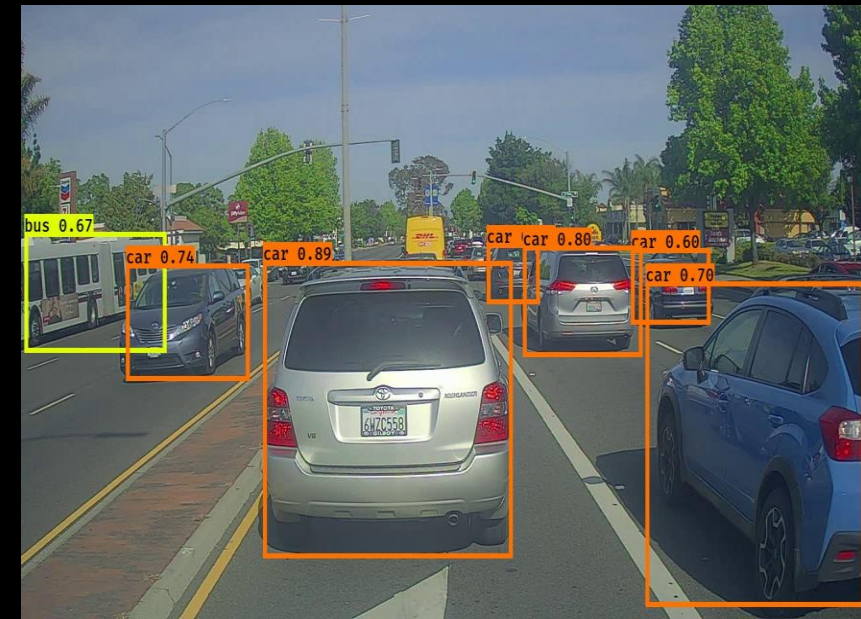
Image Classification



Classification with Localization



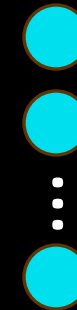
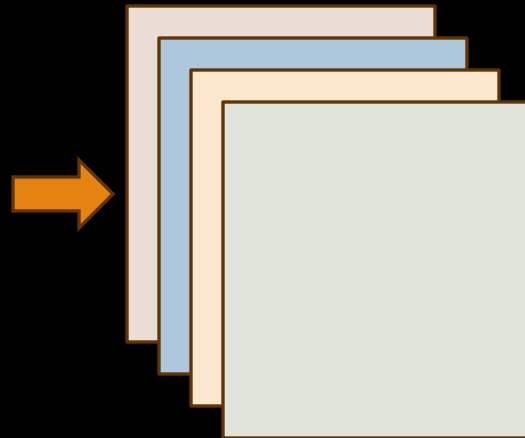
Detection



Object Detection

Object Localization

Classification with localization



Softmax (4)

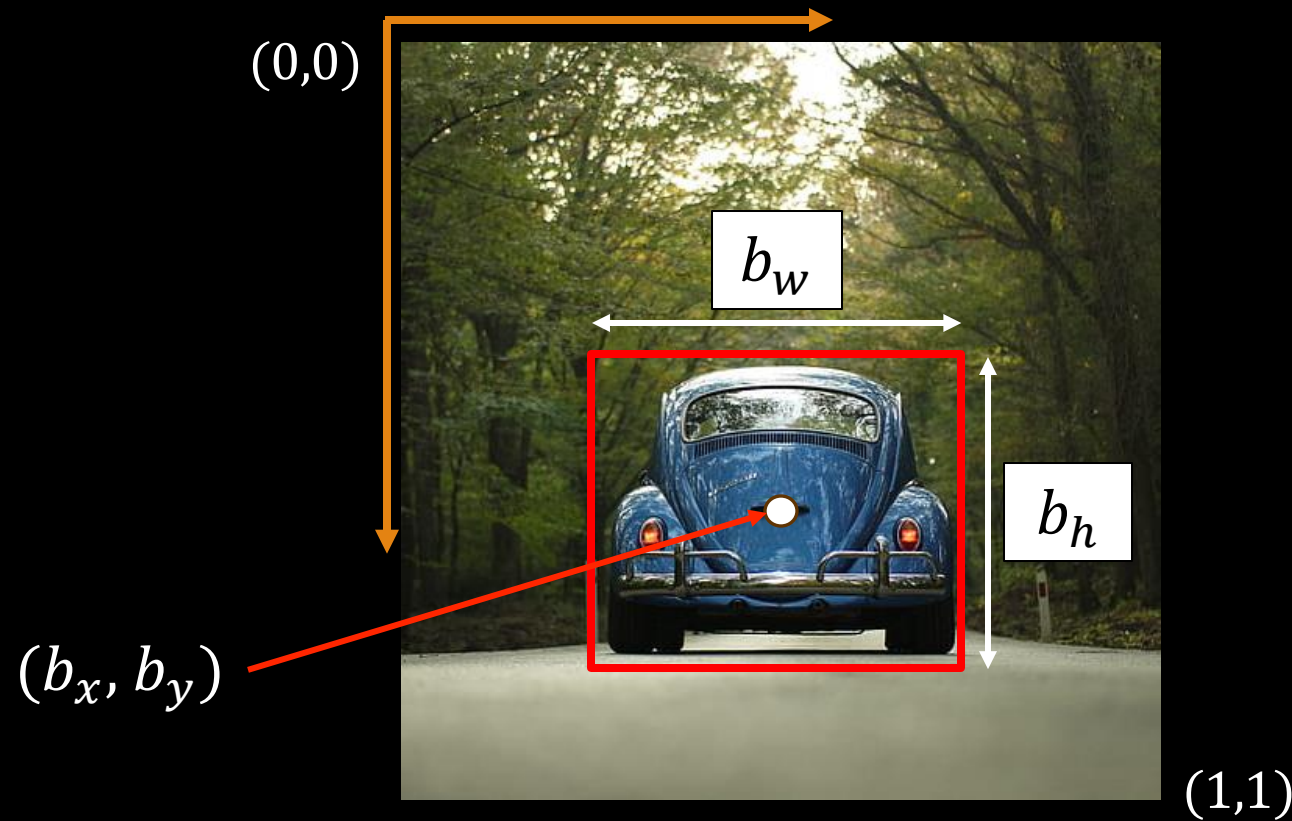
b_x, b_y, b_h, b_w

- 1 – pedestrian
- 2 – car
- 3 – motorcycle
- 4 – background

Object Detection

Object Localization

Classification with localization



$$\begin{aligned}b_x &= 0.5 \\b_y &= 0.7 \\b_h &= 0.3 \\b_w &= 0.4\end{aligned}$$

Object Detection

Defining the target label y

1 – pedestrian

2 – car

3 – motorcycle

4 – background

Need to output b_x, b_y, b_h, b_w , class label (1 – 4)

$X =$



$$y = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$



$$y = \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

$$y = \begin{bmatrix} P_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Log. Reg. Loss

MSE

Softmax

$$\text{If } P_c = 1, \quad \text{Loss} = \sum_{i=1}^8 (\hat{y}_i - y_i)^2$$

$$\text{If } P_c = 0, \quad \text{Loss} = (\hat{y}_1 - y_1)^2$$



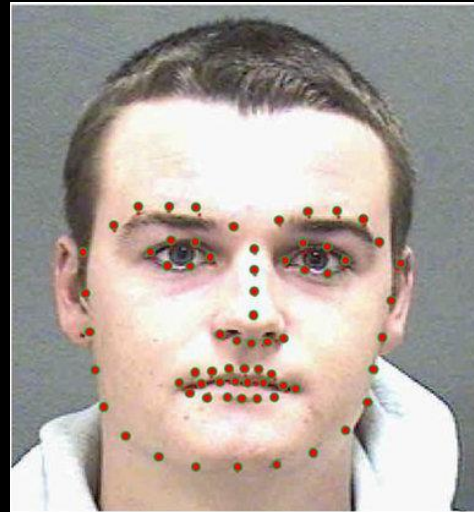
Here, we used squared error function

Object Detection

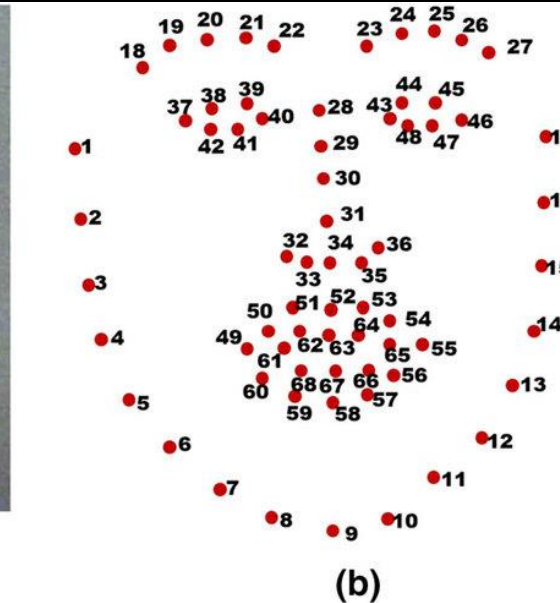
Landmark Detection



b_x, b_y, b_h, b_w

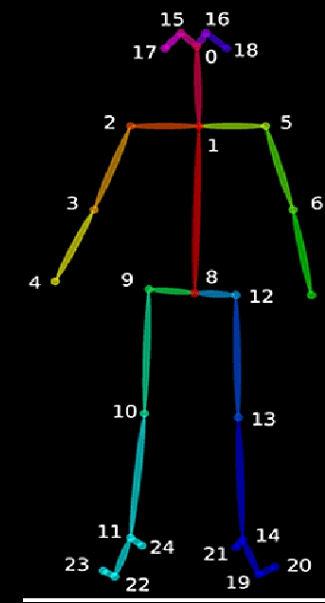


(a)



(b)

$l_1, l_2 \dots l_{68}$



$l_0, l_1 \dots l_{24}$



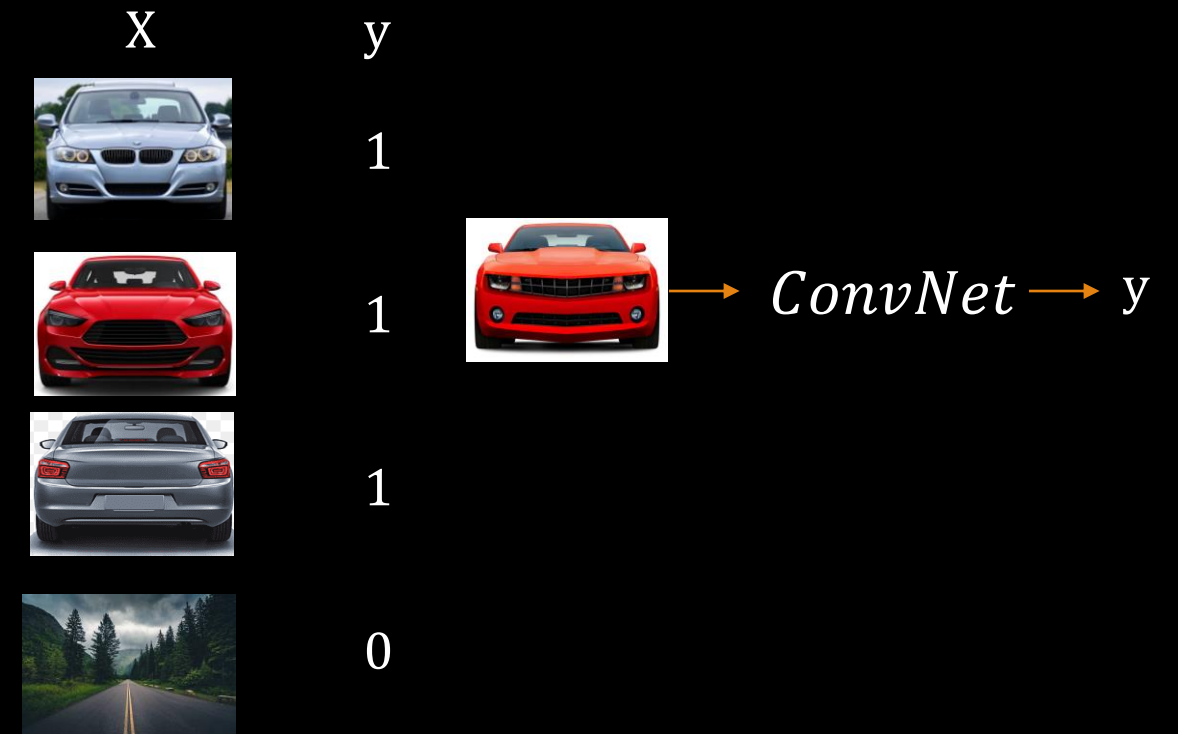
a)

Object Detection

Car detection Example

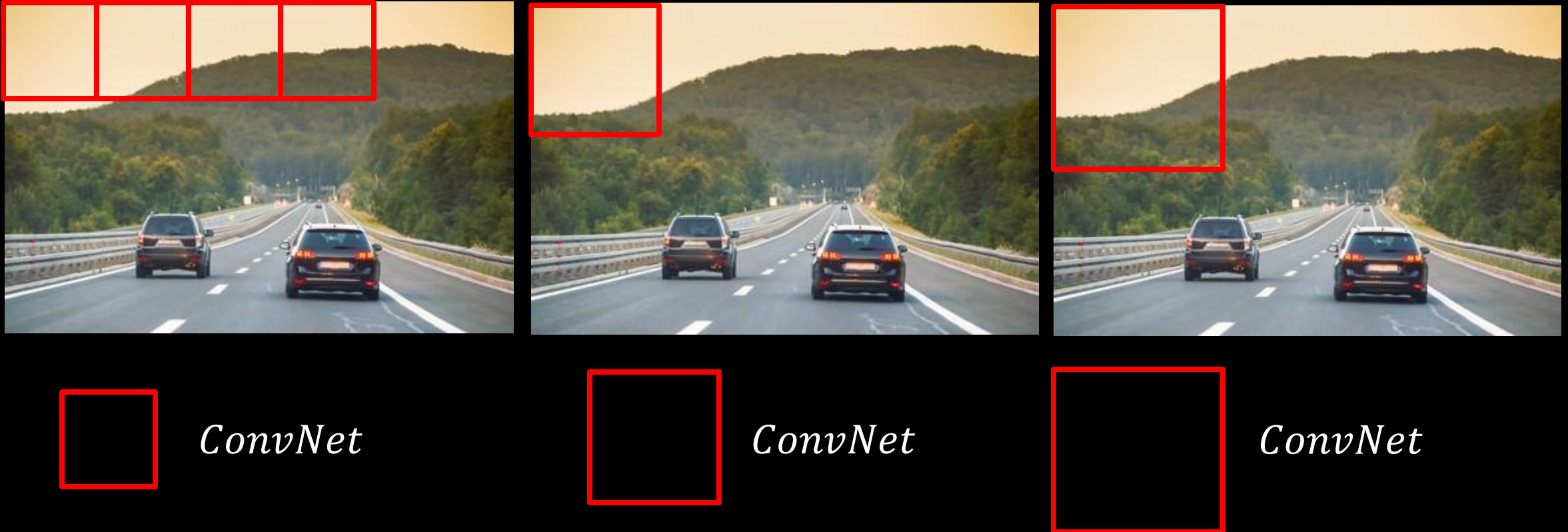


Training set:



Object Detection

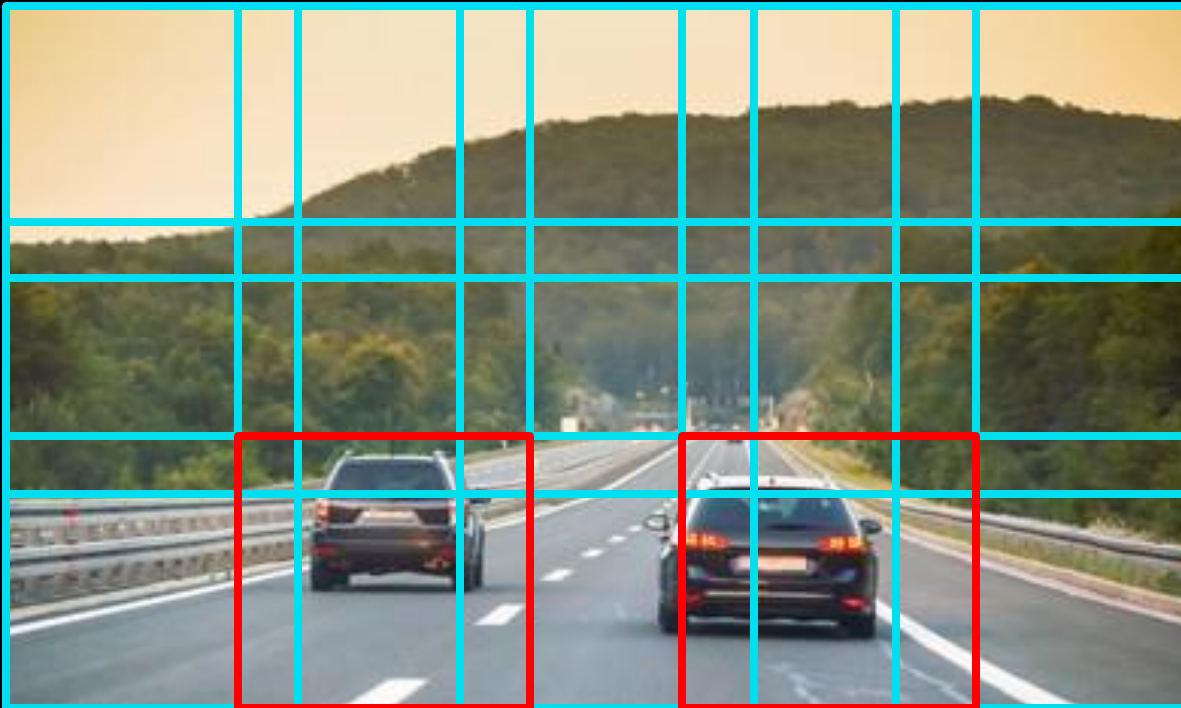
Sliding Windows Detection



The biggest disadvantage is computational cost

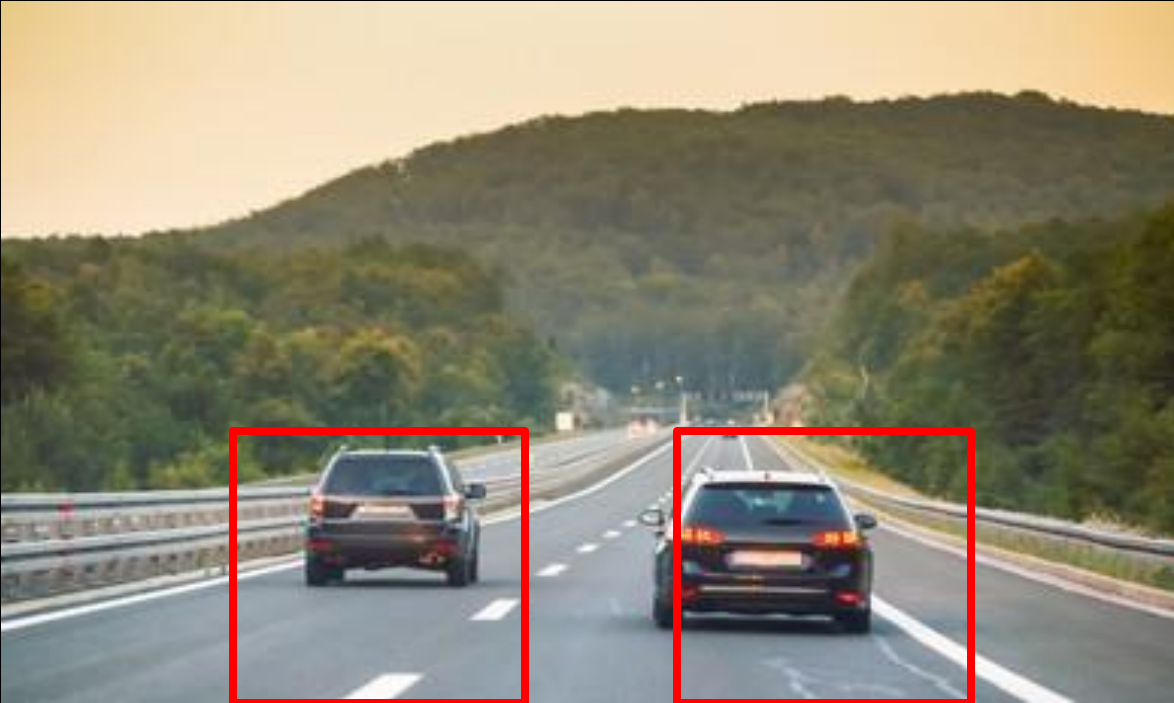
Object Detection

ConvNet implementation of Sliding Windows



Object Detection

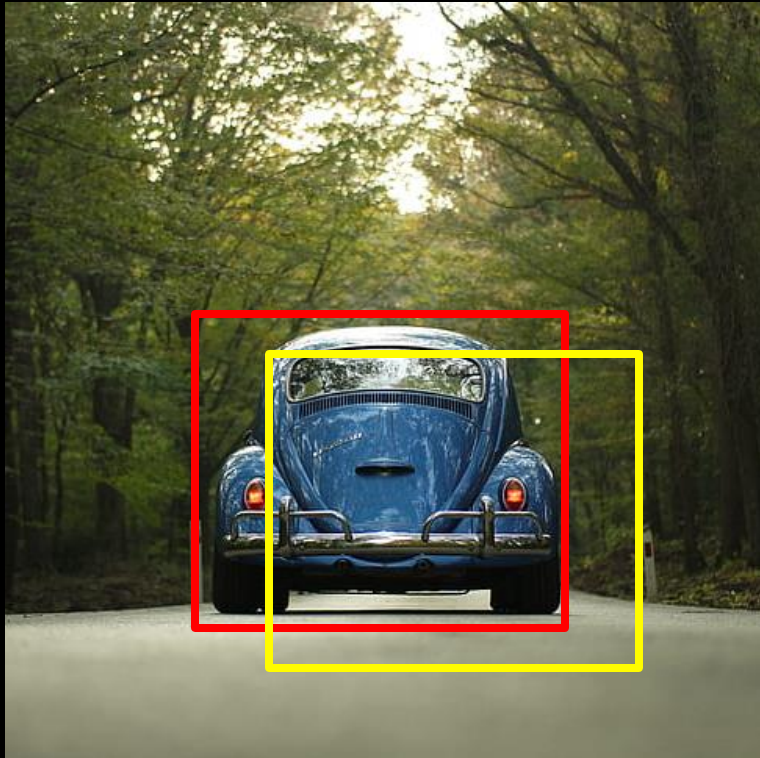
ConvNet implementation of Sliding Windows



Weakness of this method is that the bounding boxes coordinates are **not too accurate.**

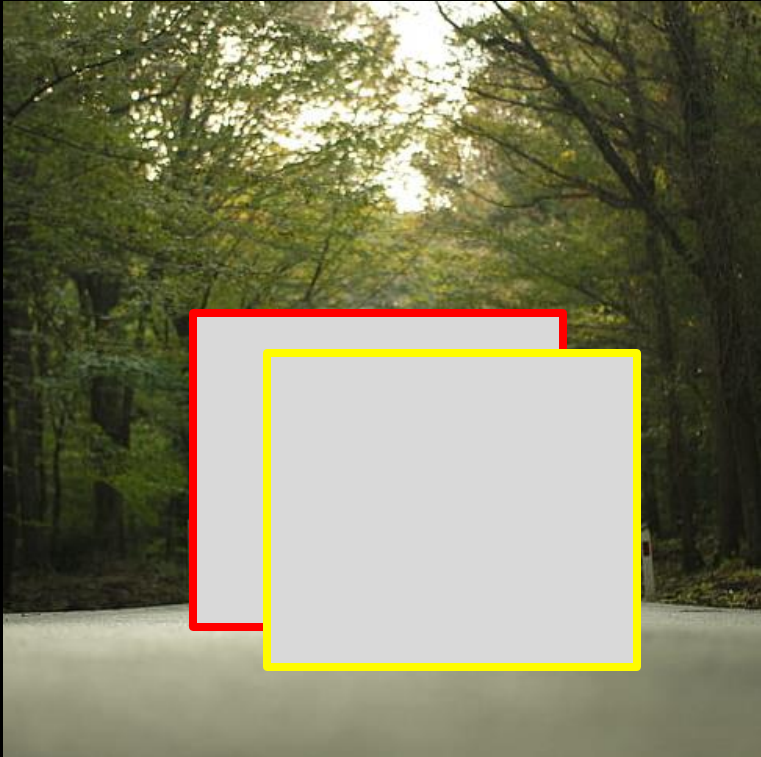
Object Detection

Intersection over Union (IoU)



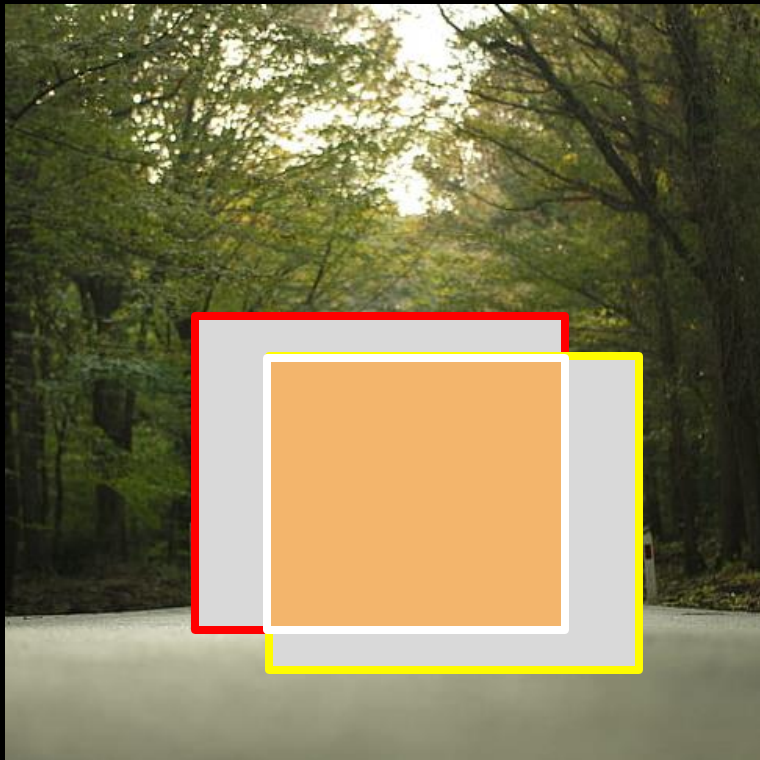
Object Detection

Intersection over Union (IoU)



Object Detection

Intersection over Union (IoU)



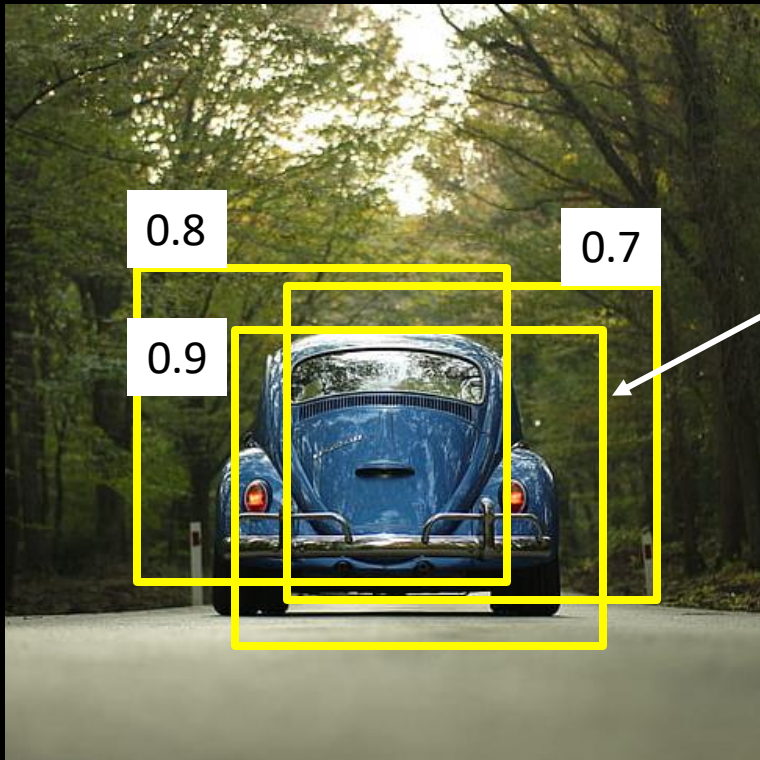
$$IoU = \frac{\text{Size of } \boxed{\text{orange}}}{\text{Size of } \boxed{\text{gray}}}$$

"Correct" if $IoU \geq 0.5$

Generally, **IoU** is a measure of the overlap between two bounding boxes

Object Detection

Non – max suppression

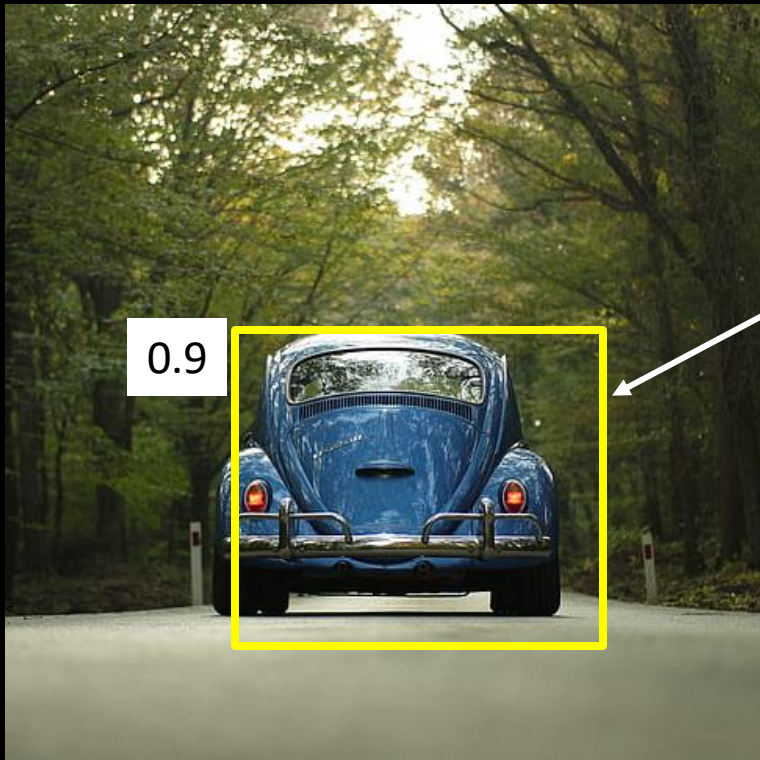


P_c

Non-Maximum Suppression (NMS) is a post-processing technique used in object detection to remove **duplicate** bounding boxes that detect the **same object**. Its purpose is to keep only the **best bounding box** for each detected object.

Object Detection

Non – max suppression

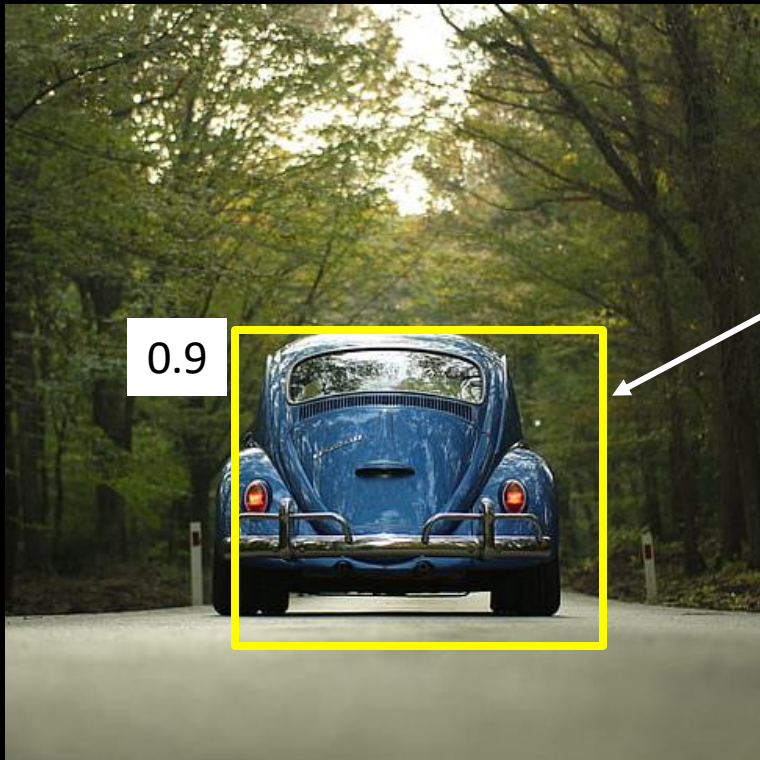


P_c

Non-Maximum Suppression (NMS) is a post-processing technique used in object detection to remove **duplicate** bounding boxes that detect the **same object**. Its purpose is to keep only the **best bounding box** for each detected object.

Object Detection

Non – max suppression



P_c

While there are any remaining boxes:

- Pick the box with largest P_c output that as a prediction
- Discard any remaining box with $IoU \leq 0.5$

Object Detection

Two Main Categories Object detection models

A. Two-Stage Detectors

These models detect objects in **two steps**:

1. **Generate region proposals** (candidate object locations)
2. **Classify + refine bounding boxes** for each proposal

✦ Examples:

- R-CNN
- Fast R-CNN
- Faster R-CNN
- Mask R-CNN (adds segmentation)

✓ Pros:

- High accuracy
- Good for small objects

✗ Cons:

- Slower (not ideal for real-time)

B. One-Stage Detectors

These models detect objects in a **single step**:

Directly predict **class + bounding box** from the image.

✦ Examples:

- YOLO family (YOLOv1...YOLOv9)
- SSD (Single Shot Detector)
- RetinaNet

✓ Pros:

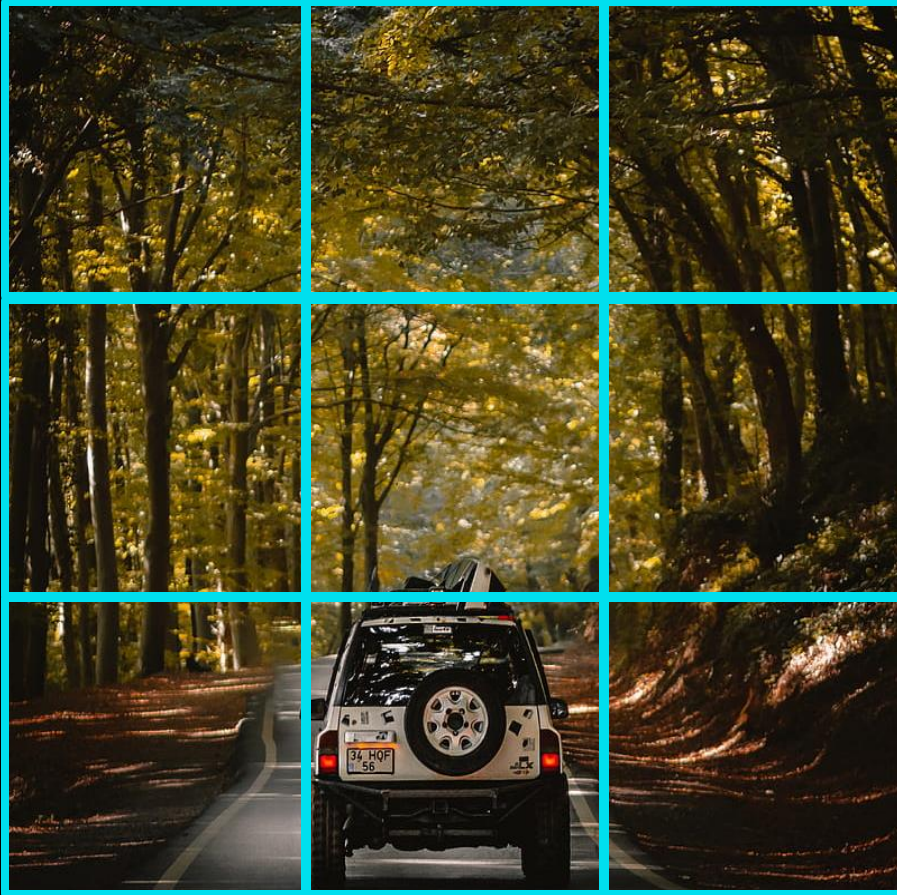
- Very fast (real-time)
- Efficient

✗ Cons:

- Historically less accurate (but closing gap)

Object Detection

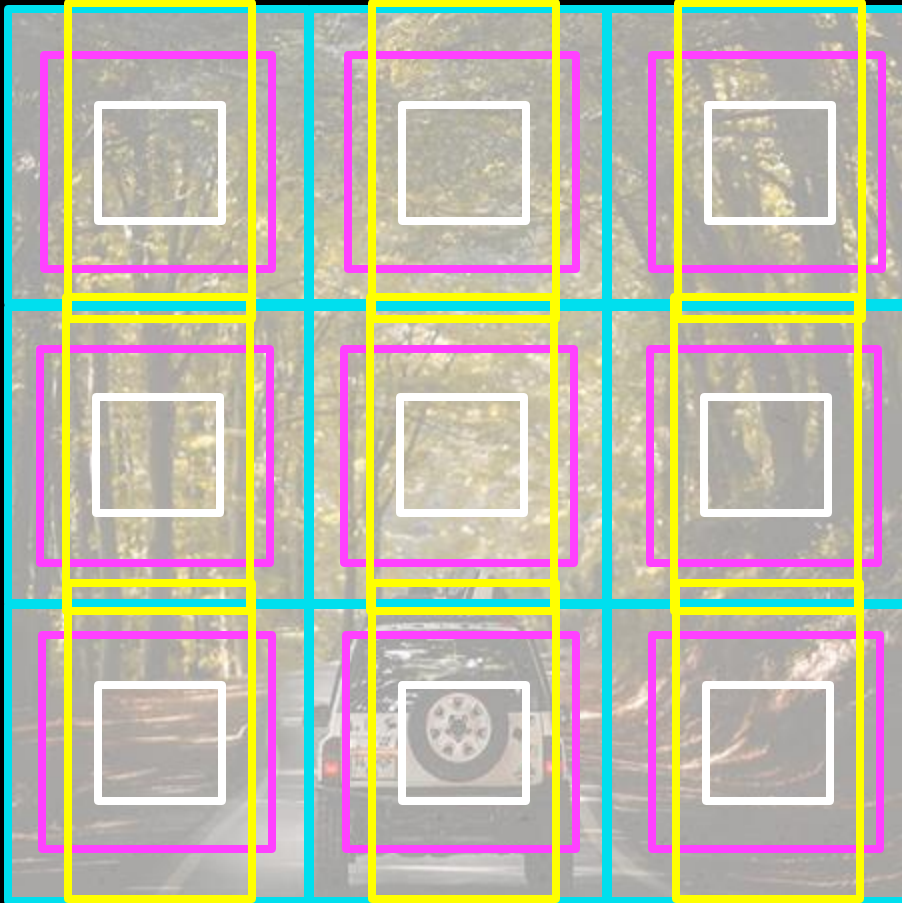
Anchor Boxes (YOLO)



- **Definition:** An anchor box is a predefined rectangle with a specific size (height, width) and aspect ratio (ratio of width to height).
- **Multiple anchor boxes** are defined for each location in the image grid.

Object Detection

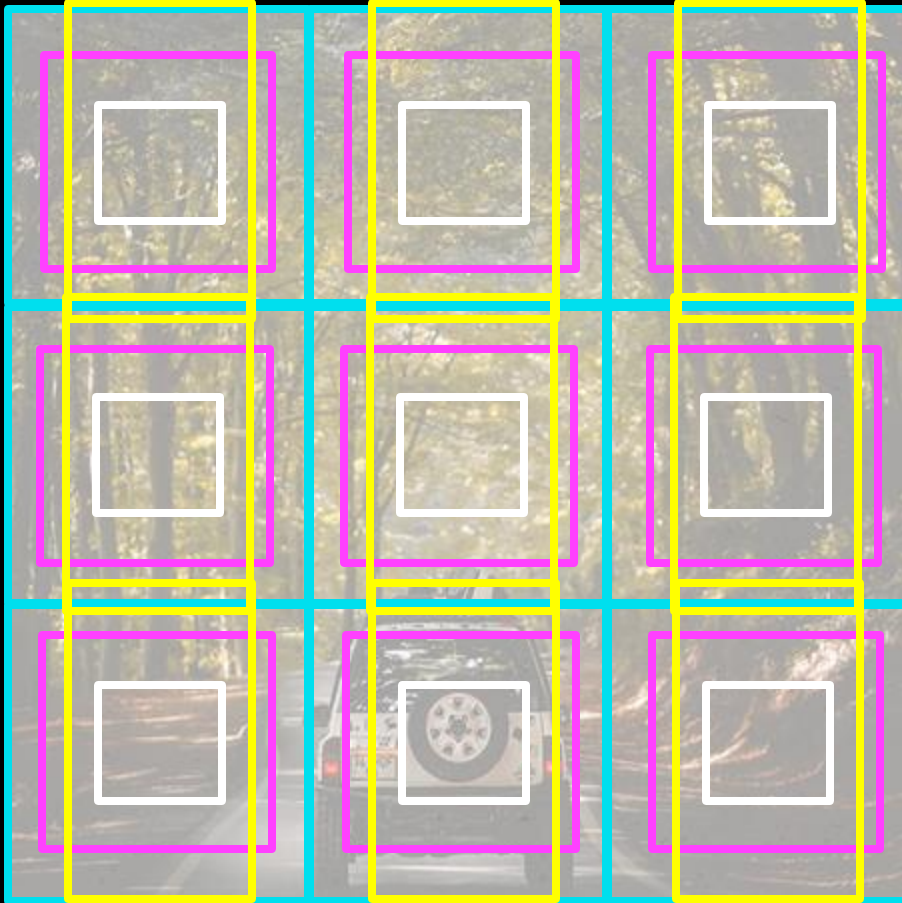
Anchor Boxes (YOLO)



- **Definition:** An anchor box is a predefined rectangle with a specific size (height, width) and aspect ratio (ratio of width to height).
- **Multiple anchor boxes** are defined for each location in the image grid.

Object Detection

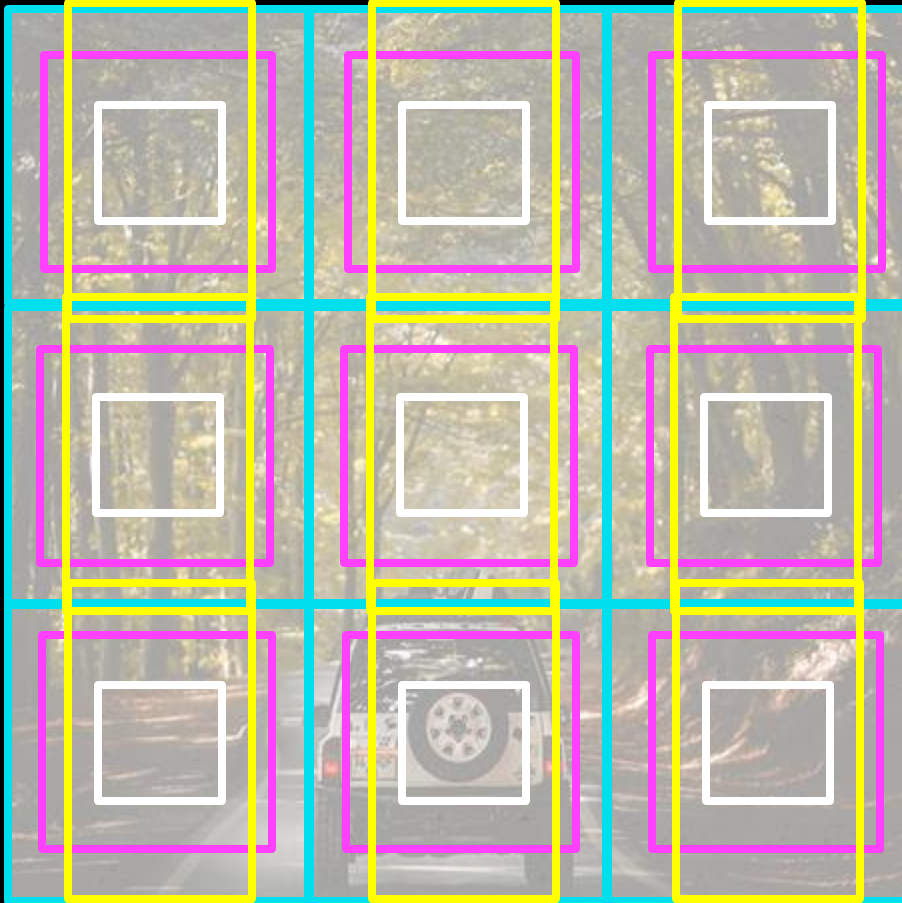
Anchor Boxes (YOLO)



- **Why Used?:** Real-world objects vary greatly in shape, size, and aspect ratio. Anchor boxes help object detection models predict bounding boxes for objects more effectively by providing a starting point for predictions.

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

1. Predefined Boxes:

1. Anchor boxes are designed before training and are not learned during training.
2. Each grid cell in the feature map has multiple anchor boxes associated with it, often with different scales and aspect ratios.

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

2. Assigning Ground Truth:

During training, the algorithm assigns each ground truth box to the most appropriate anchor box based on the **Intersection over Union (IoU)** between them.

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

3. Prediction:

1. The model predicts the **offsets** (shifts) and **scales** (resizing factors) required to adjust the anchor boxes to match the ground truth boxes for the detected objects.
2. Additionally, it predicts a **confidence score** and **class label** for each anchor box.

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

 → Ground truth

 → Anchor box 1

 → Anchor box 2

 → Anchor box 3

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

4. Post-Processing:

1. After prediction, anchor boxes with low confidence scores are filtered out.
2. **Non-Maximum Suppression (NMS)** is applied to remove duplicate or overlapping predictions for the same object.

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

-  → Ground truth
 -  → Anchor box 1
 -  → Anchor box 2
 -  → Anchor box 3
- 1 – pedestrian
2 – car
3 – motorcycle
4 – background

$$y = [P_c t_x t_y t_h t_w c_1 c_2 c_3 \quad P_c t_x t_y t_h t_w c_1 c_2 c_3 \quad P_c t_x t_y t_h t_w c_1 c_2 c_3]$$

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

-  → Ground truth
 -  → Anchor box 1
 -  → Anchor box 2
 -  → Anchor box 3
- 1 – pedestrian
2 – car
3 – motorcycle
4 – background

$$y = [0.67t_x t_y t_h t_w 0 10 \quad 0.73t_x t_y t_h t_w 0 10 \quad 0.49t_x t_y t_h t_w 0 10]$$

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

 → Ground truth

 → Anchor box 1 $\text{IoU}=0.75$

 → Anchor box 2 $\text{IoU}=0.80$

 → Anchor box 3 $\text{IoU}=0.45$

$$y = [\underbrace{0.67t_x t_y t_h t_w}_{\text{Anchor box 1}} \underbrace{010}_{\text{Class}} \underbrace{0.73t_x t_y t_h t_w}_{\text{Anchor box 2}} \underbrace{010}_{\text{Class}} \underbrace{0.79t_x t_y t_h t_w}_{\text{Anchor box 3}} \underbrace{010}_{\text{Class}}]$$

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

 → Ground truth

 → Anchor box 2 $\text{IoU}=0.80$

$$y = [0.73 \overbrace{t_x t_y t_h t_w}^{\text{Anchor box 2}} 0 1 0]$$

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

$$y = [0.73 \ t_x \ t_y \ t_h \ t_w \ 0 \ 1 \ 0]$$

Calculating Bounding box coordinates:

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

$$b_h = p_h e^{t_h}$$

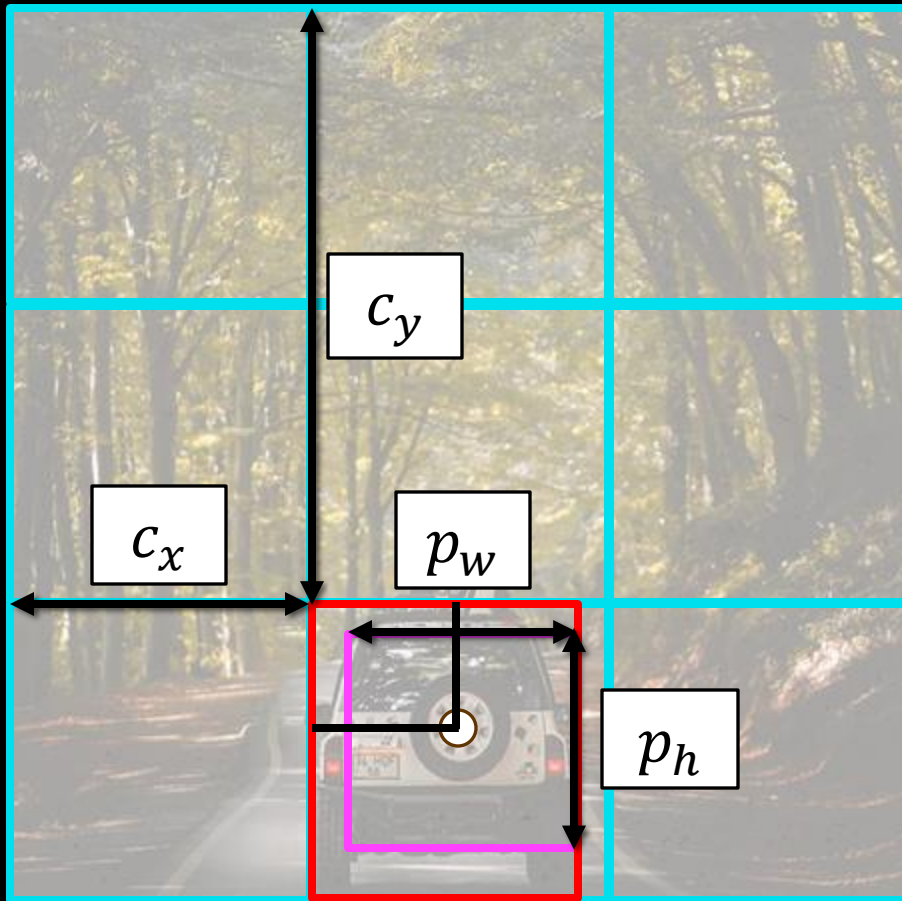
$$b_w = p_w e^{t_w}$$

c_x, c_y : Top-left corner of the grid cell.

p_w, p_h : Width and height of the anchor box.

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

$$y = [0.73 \ t_x \ t_y \ t_h \ t_w \ 0 \ 1 \ 0]$$

Calculating Bounding box coordinates:

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

$$b_h = p_h e^{t_h}$$

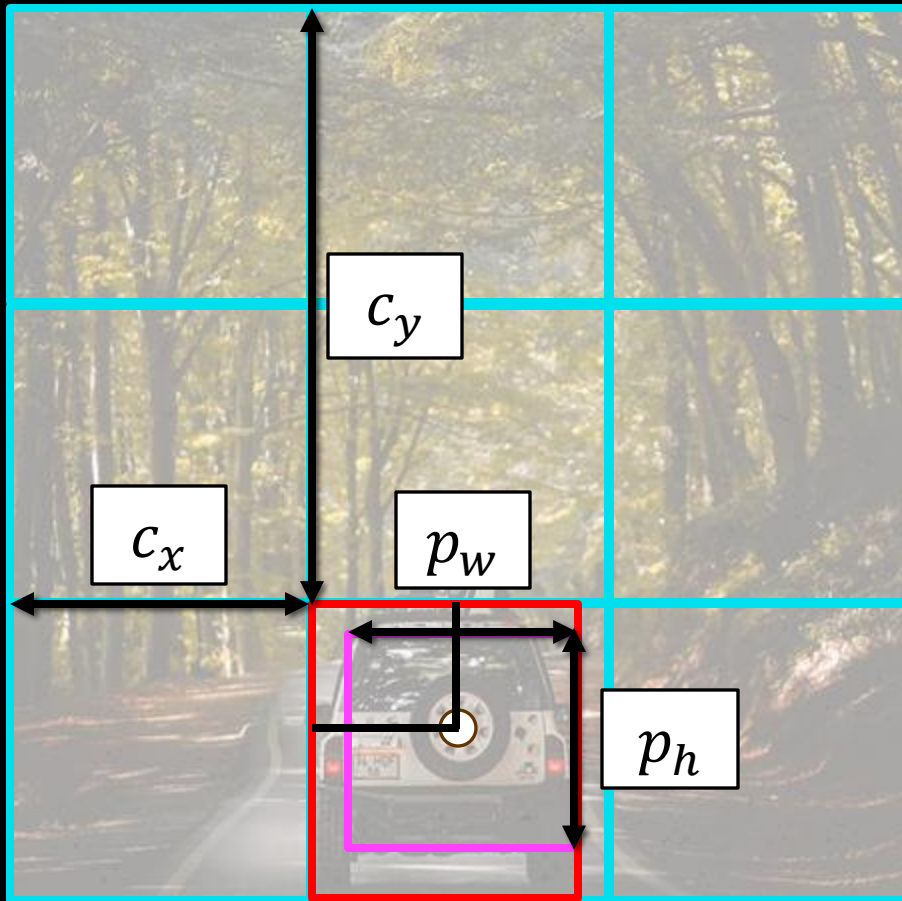
$$b_w = p_w e^{t_w}$$

c_x, c_y : Top-left corner of the grid cell.

p_w, p_h : Width and height of the anchor box.

Object Detection

Anchor Boxes (YOLO)



How Anchor Boxes Work in Object Detection

$$\hat{y} = [P_c \ t_x \ t_y \ t_h \ t_w \ c_1 \ c_2 \ c_3]$$

$$y = [P_c \ b_x \ b_y \ b_h \ b_w \ c_1 \ c_2 \ c_3]$$

Converting Bounding box coordinates:

$$t_x = b_x - c_x$$

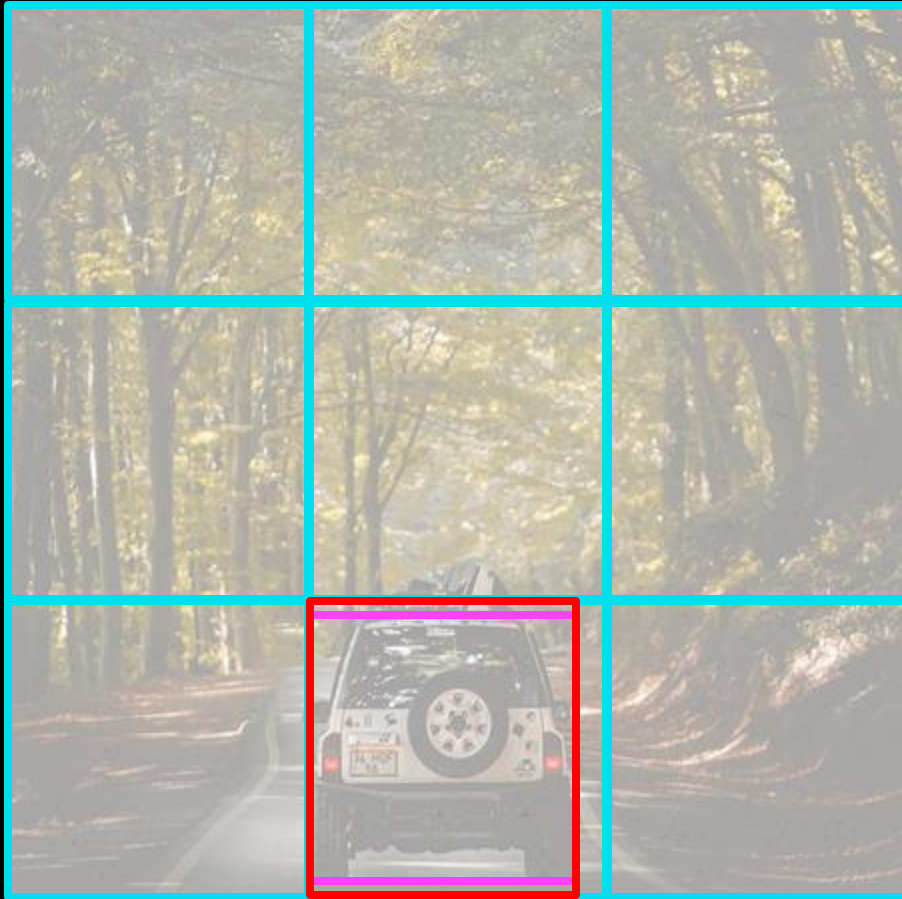
$$t_y = b_y - c_y$$

$$t_h = \log\left(\frac{b_h}{p_h}\right)$$

$$t_w = \log\left(\frac{b_w}{p_w}\right)$$

Object Detection

Anchor Boxes (YOLO)



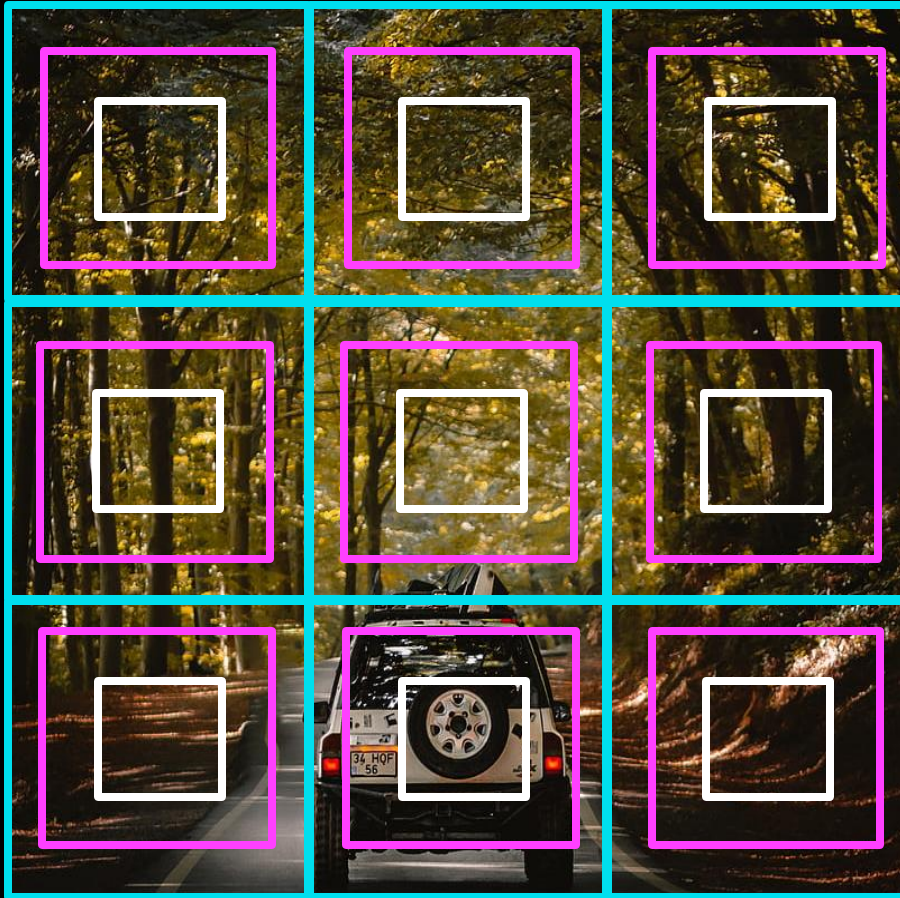
How Anchor Boxes Work in Object Detection

$$y = [0.73 \ t_x \ t_y \ t_h \ t_w \ 0 \ 1 \ 0]$$

Final bounding box for object.

Object Detection

You Only Look Once (YOLO) Algorithm



- 1 – pedestrian
- 2 – car
- 3 – motorcycle

$$y = 3 \times 3 \times 2 \times 8$$

3X3 is grid size
2 is # anchors
8 is P, box coordinates
and #classes

$$y = \begin{bmatrix} P_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ P_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \quad \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix} \quad \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Object Detection

Anchor Boxes

Functions of Anchor Boxes

1. Multi-Scale Object Detection:

- Anchor boxes allow the detection of objects at multiple scales by associating different sizes and aspect ratios with grid cells in the feature map.
- This is especially useful for detecting small and large objects in the same image.

2. Handling Aspect Ratios:

- Objects in an image can have different shapes (e.g., tall, wide, square). By using anchor boxes with varied aspect ratios, the model can better accommodate these variations.

Object Detection

Anchor Boxes

Functions of Anchor Boxes

3. Prediction Efficiency:

- Instead of predicting bounding boxes from scratch, the model predicts adjustments to predefined anchor boxes, simplifying the learning process.

4. Flexibility in Localization:

- Anchor boxes provide a systematic way to divide the search space, ensuring that each grid cell can potentially detect multiple objects.

Object Detection

Anchor Boxes

Where Are Anchor Boxes Used?

- **Single Shot Detector (SSD):**

- SSD uses anchor boxes (called **default boxes**) at different scales for detecting objects at multiple resolutions.

- **Faster R-CNN:**

- Faster R-CNN uses anchor boxes in its Region Proposal Network (RPN) to generate candidate regions of interest.

- **YOLO (You Only Look Once):**

- YOLOv2 and later versions use anchor boxes to predict bounding boxes instead of directly regressing box coordinates.

Object Detection

Anchor Boxes

Example

Imagine detecting a dog and a car in an image:

- A grid cell might have three anchor boxes with different aspect ratios (e.g., 1:1, 2:1, 1:2).
- If the car overlaps with an anchor box of 2:1 ratio, the model adjusts this anchor box's position and size to better fit the car.
- Similarly, for the dog, the 1:1 ratio anchor box may be adjusted.

By providing these reference boxes, anchor boxes ensure that the model efficiently learns to detect and localize objects.

Object Detection

Anchor Box Algorithm

Previously:

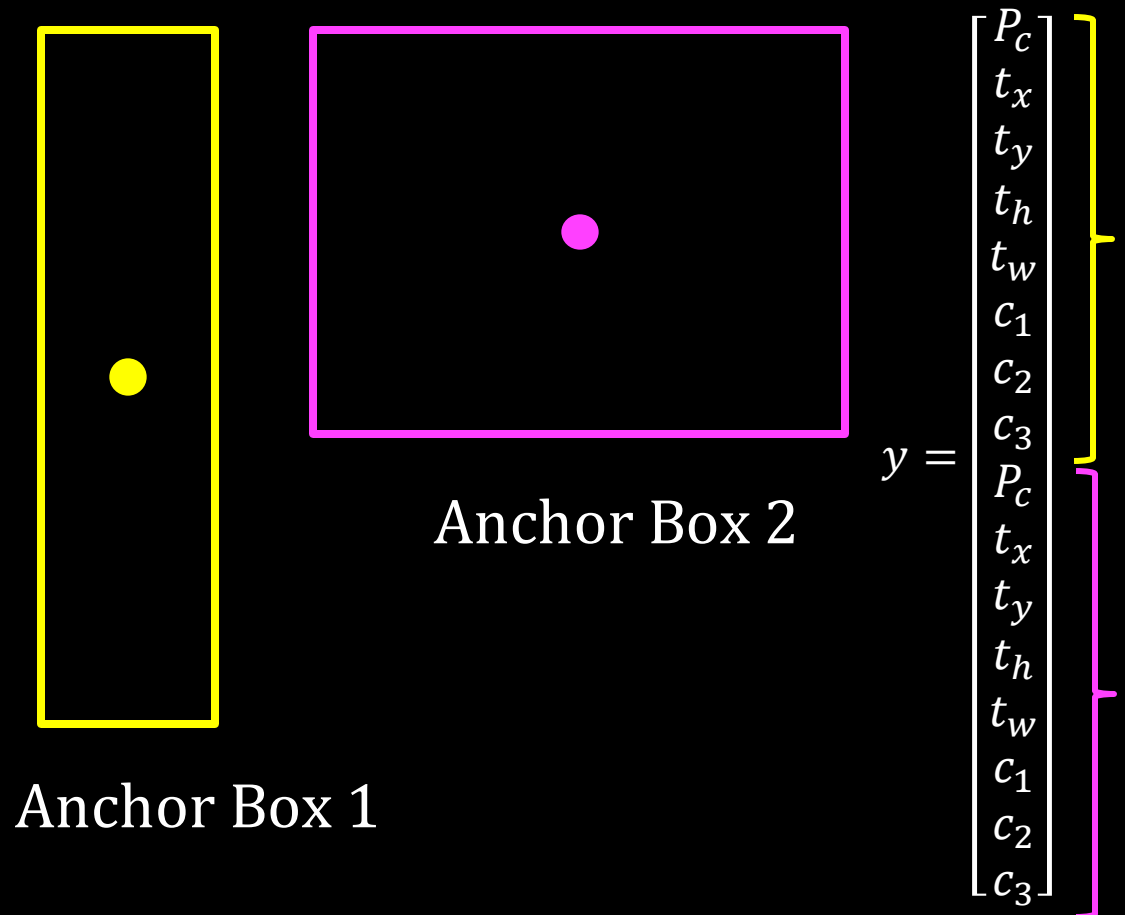
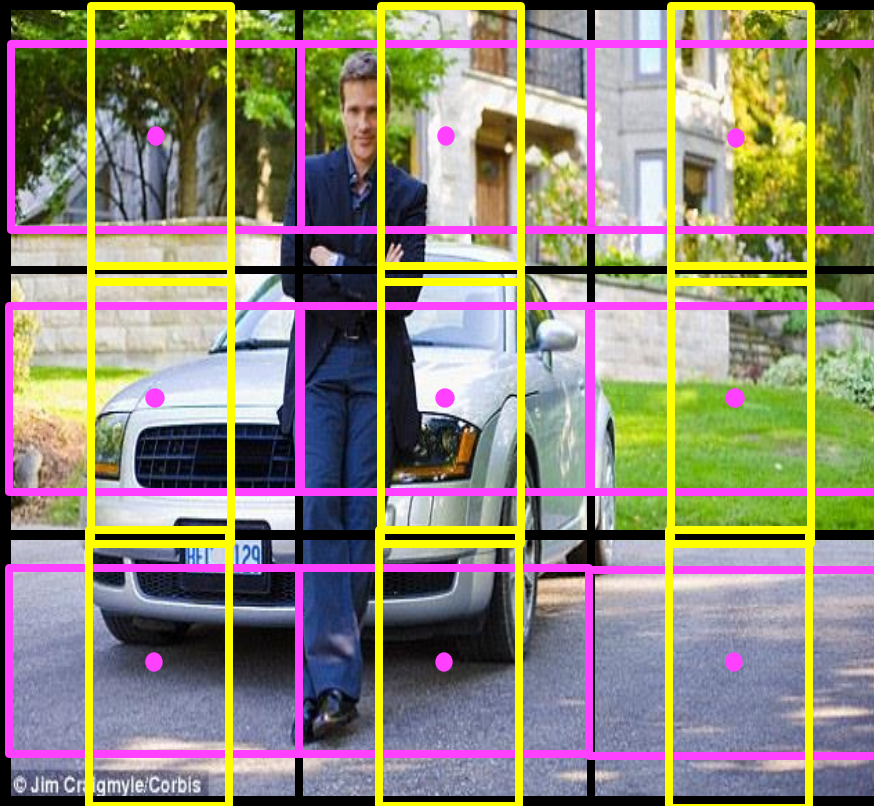
Each object in training image is assigned to grid cell that contains that object's midpoint

With two Anchor Boxes:

Each object in training image is assigned to grid cell that contains that object's midpoint and anchor box for the grid cell with highest IoU

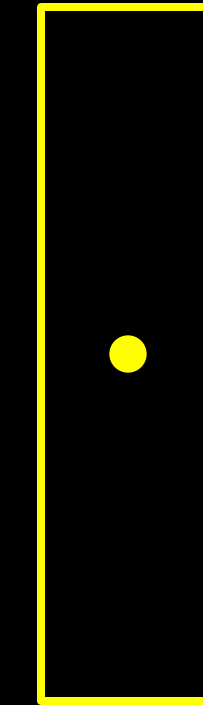
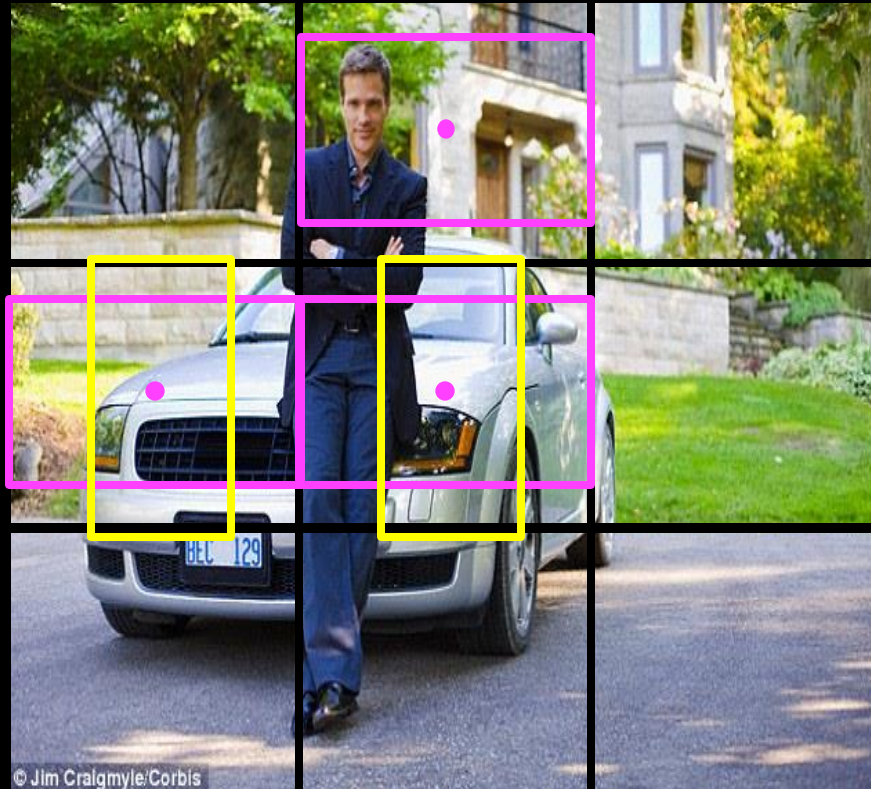
Object Detection

Anchor Boxes

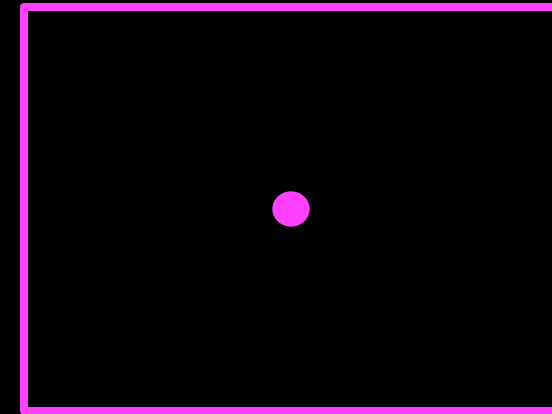


Object Detection

Anchor Boxes (After Non-max suppression)



Anchor Box 1

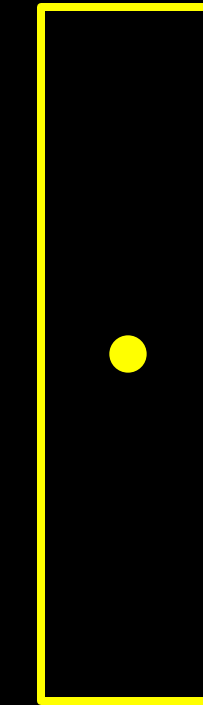
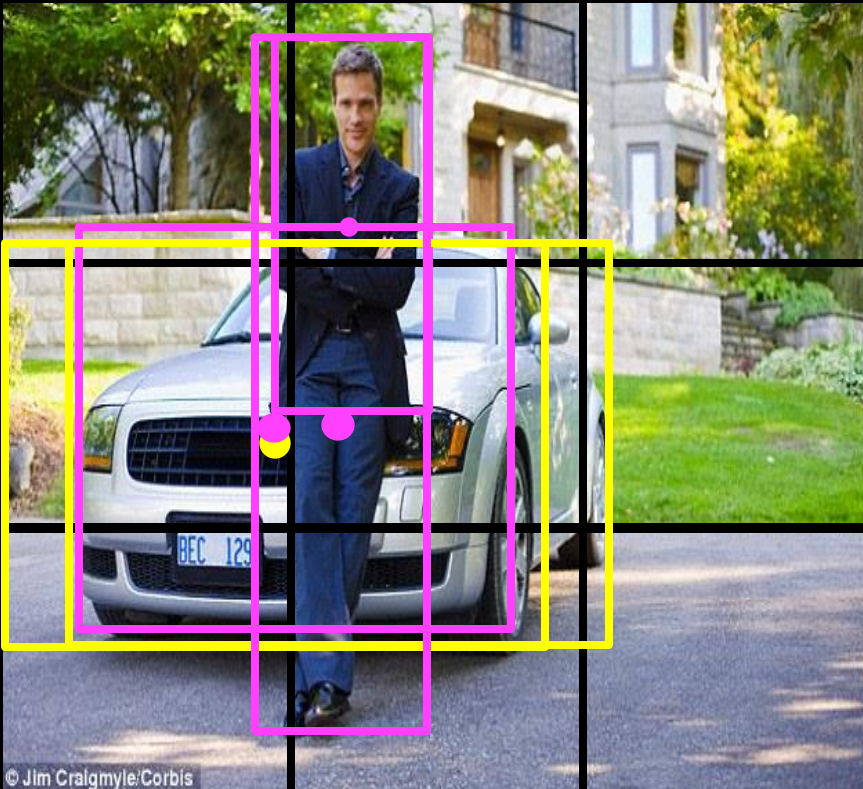


Anchor Box 2

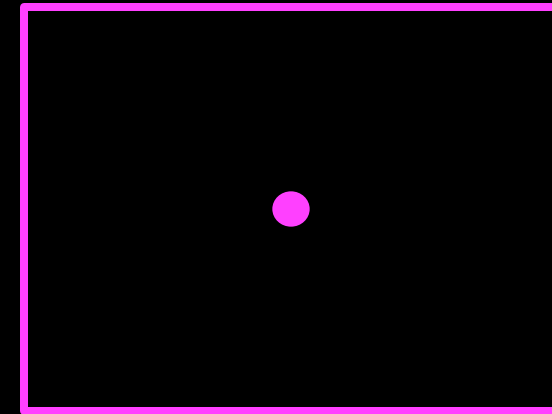
$$y = \begin{bmatrix} P_c \\ t_x \\ t_y \\ t_h \\ t_w \\ c_1 \\ c_2 \\ c_3 \\ P_c \\ t_x \\ t_y \\ t_h \\ t_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Object Detection

Anchor Boxes (After Non-max suppression)



Anchor Box 1

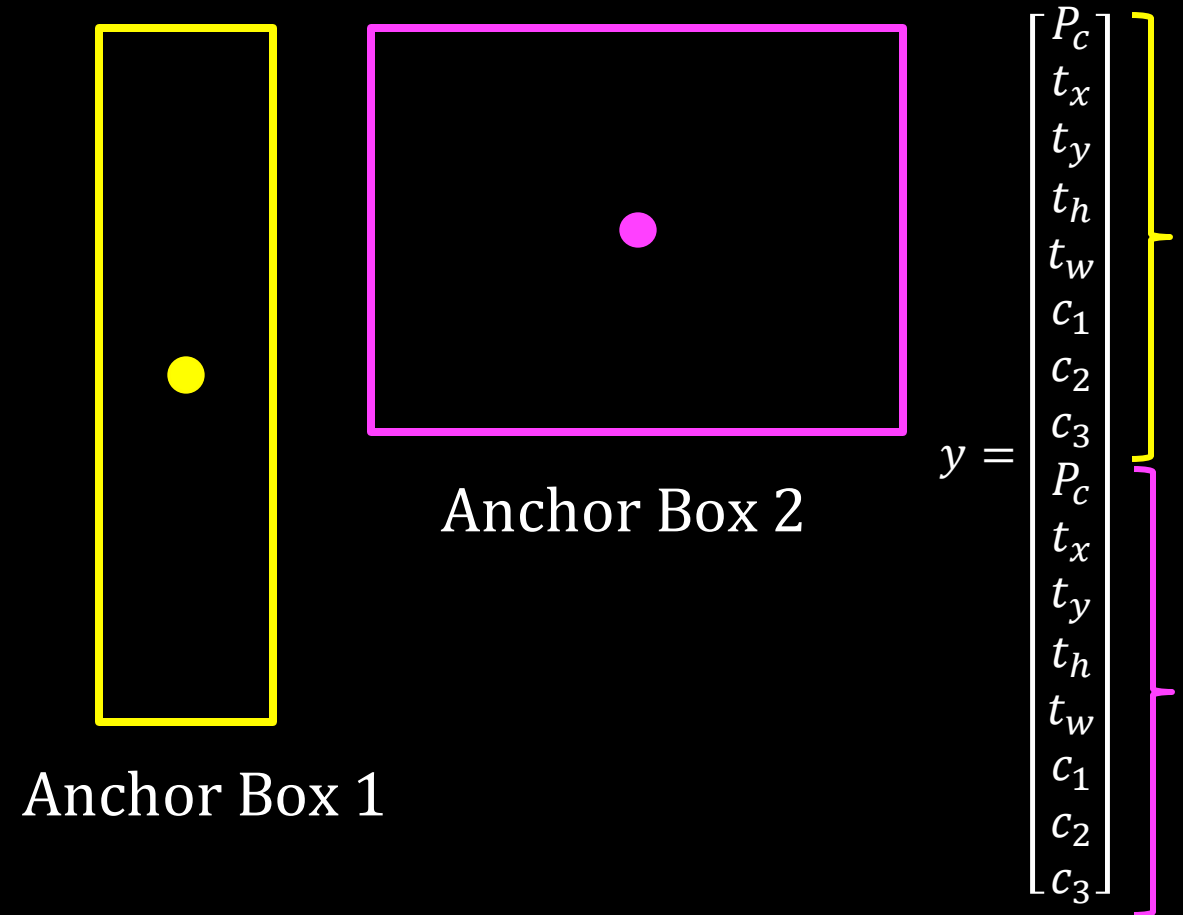
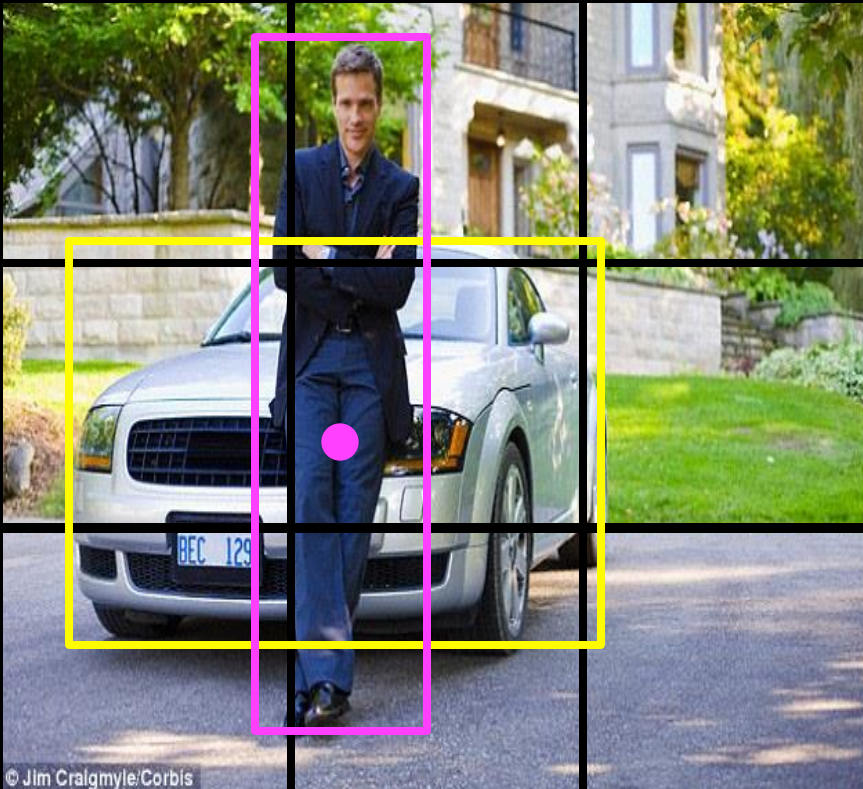


Anchor Box 2

$$y = \begin{bmatrix} P_c \\ t_x \\ t_y \\ t_h \\ t_w \\ c_1 \\ c_2 \\ c_3 \\ P_c \\ t_x \\ t_y \\ t_h \\ t_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Object Detection

Anchor Boxes

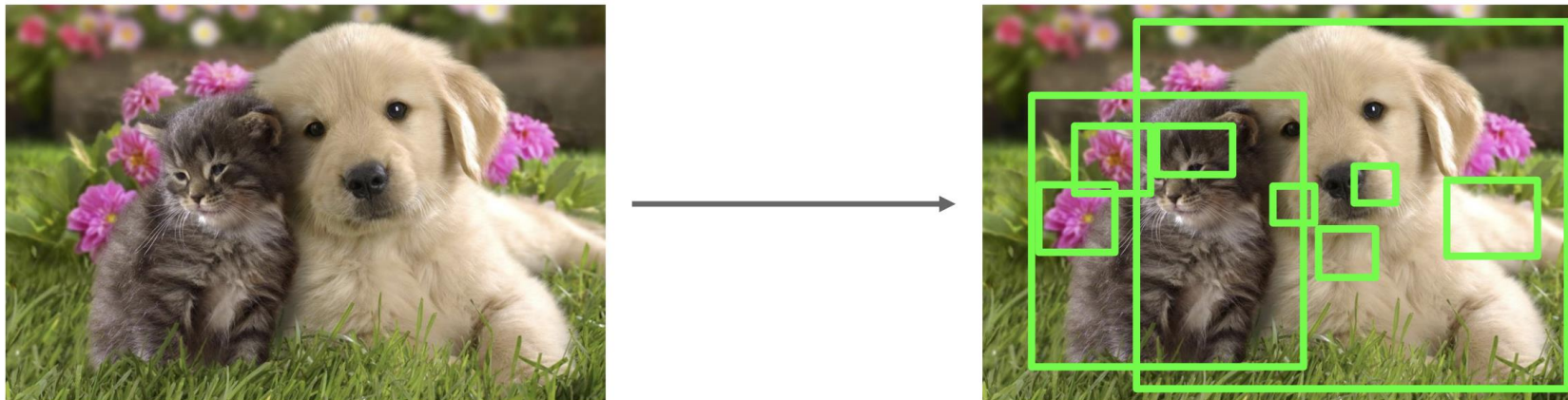


Object Detection

Regional CNN (R-CNN)

Region Proposals: Selective Search

- Find “blobby” image regions that are likely to contain objects
- Relatively fast to run; e.g. Selective Search gives 2000 region proposals in a few seconds on CPU

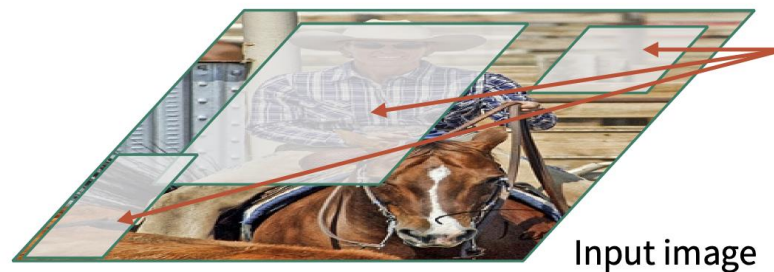


Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012
Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013
Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014
Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014

Object Detection

Regional CNN (R-CNN)

R-CNN



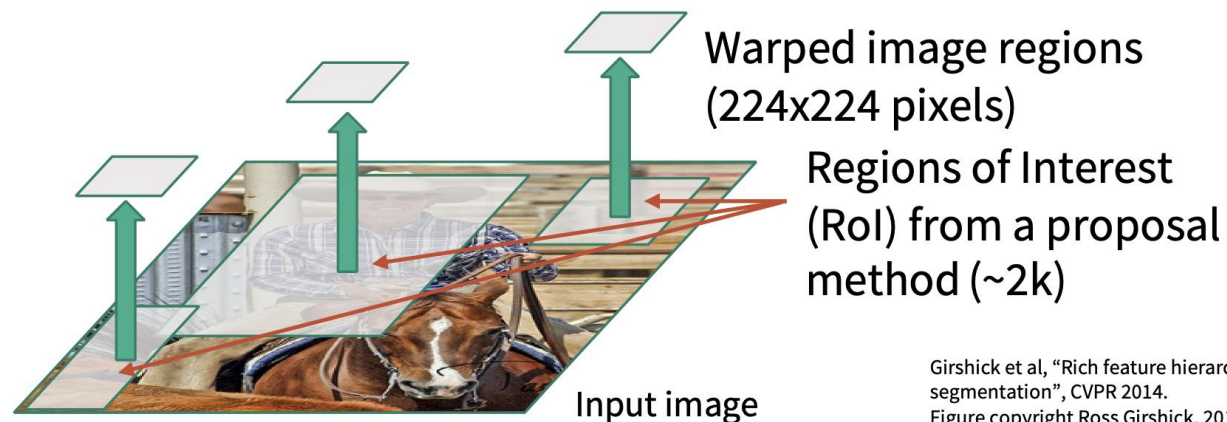
Regions of Interest
(RoI) from a proposal
method (~2k)

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Object Detection

Regional CNN (R-CNN)

R-CNN



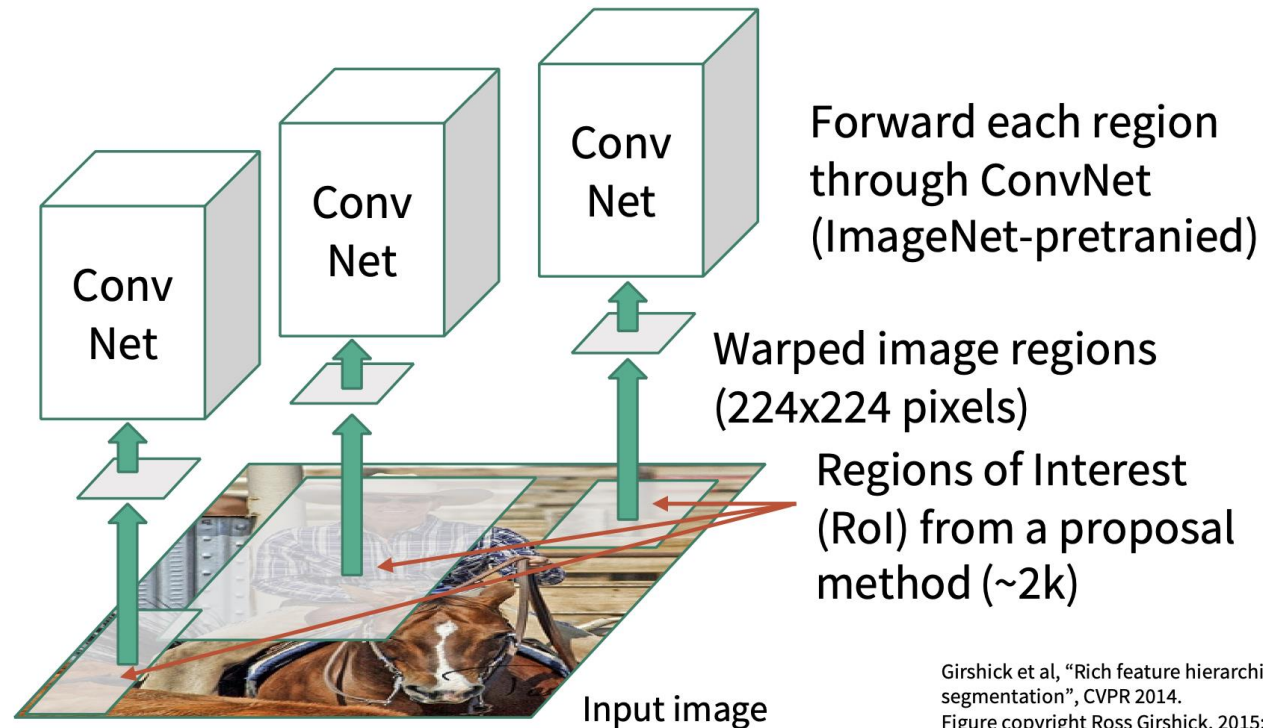
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Object Detection

Regional CNN (R-CNN)

R-CNN

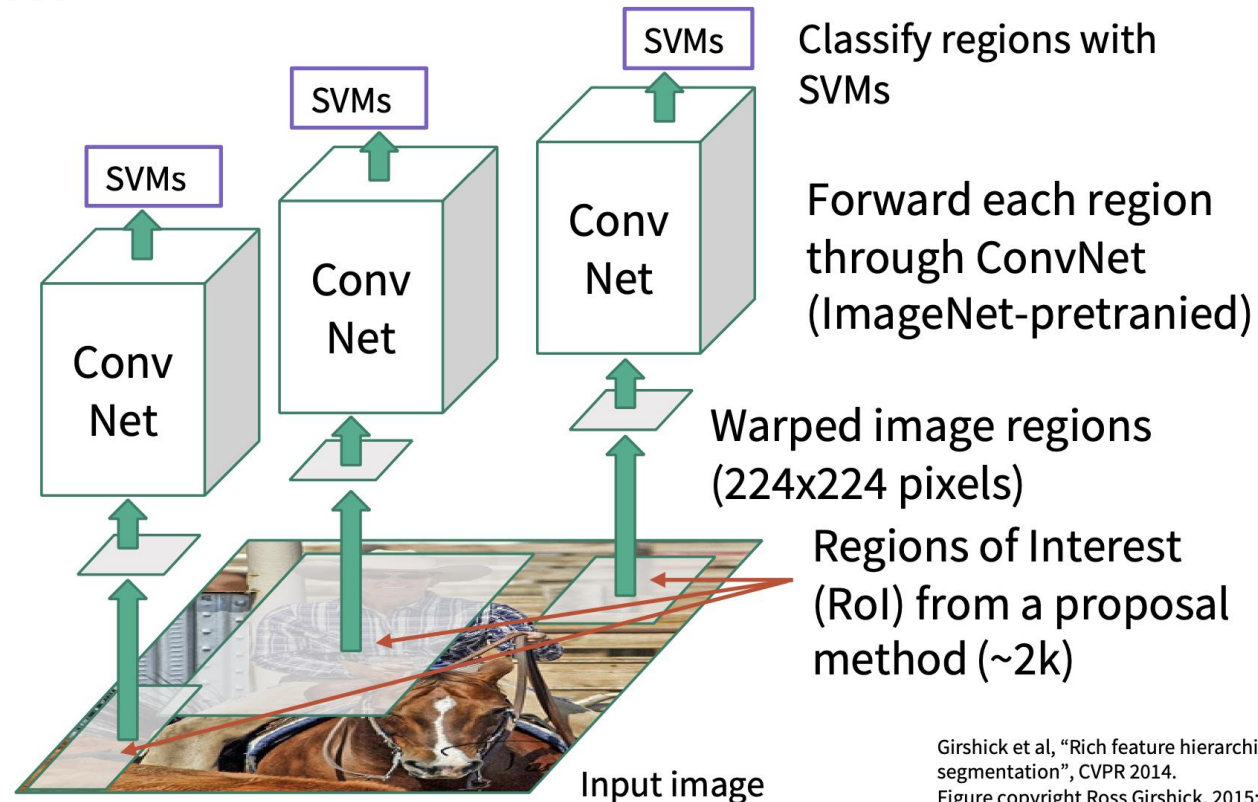


Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Object Detection

Regional CNN (R-CNN)

R-CNN

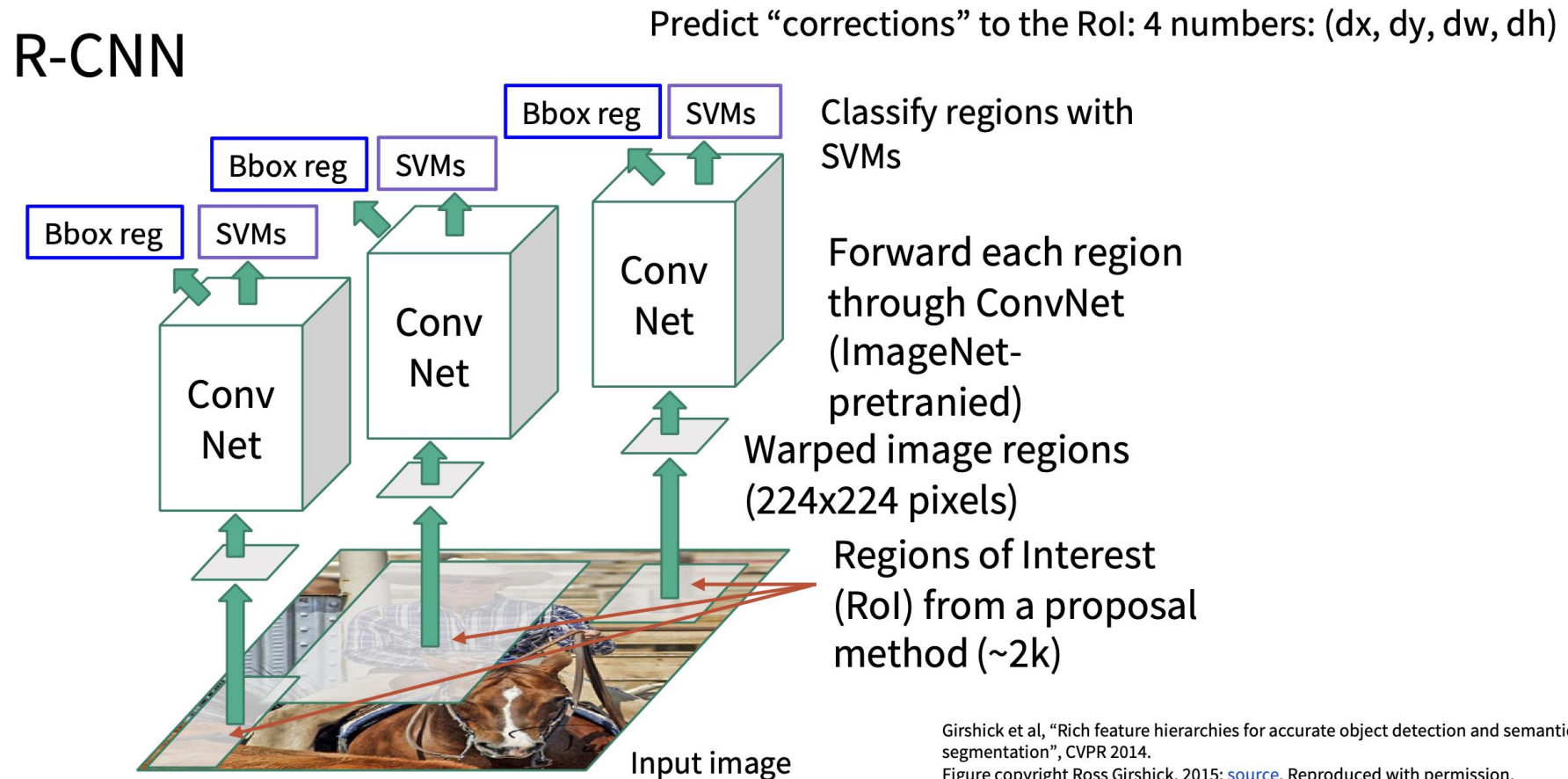


Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Object Detection

Regional CNN (R-CNN)



Girshick et al, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Object Detection

Regional CNN (R-CNN)

Calculating Bounding box coordinates:

Prediction of Regression Model for each regional proposal: d_x, d_y, d_h, d_w .

$$b_x = d_x w_{proposal} + x_{proposal}$$

$$b_y = d_y h_{proposal} + y_{proposal}$$

$$b_h = w_{proposal} e^{d_h}$$

$$b_w = h_{proposal} e^{d_w}$$

$(x_{proposal}, y_{proposal}, w_{proposal}, h_{proposal})$ are the center coordinates, width, and height of the **region proposal**.

Object Detection

Regional CNN (R-CNN)

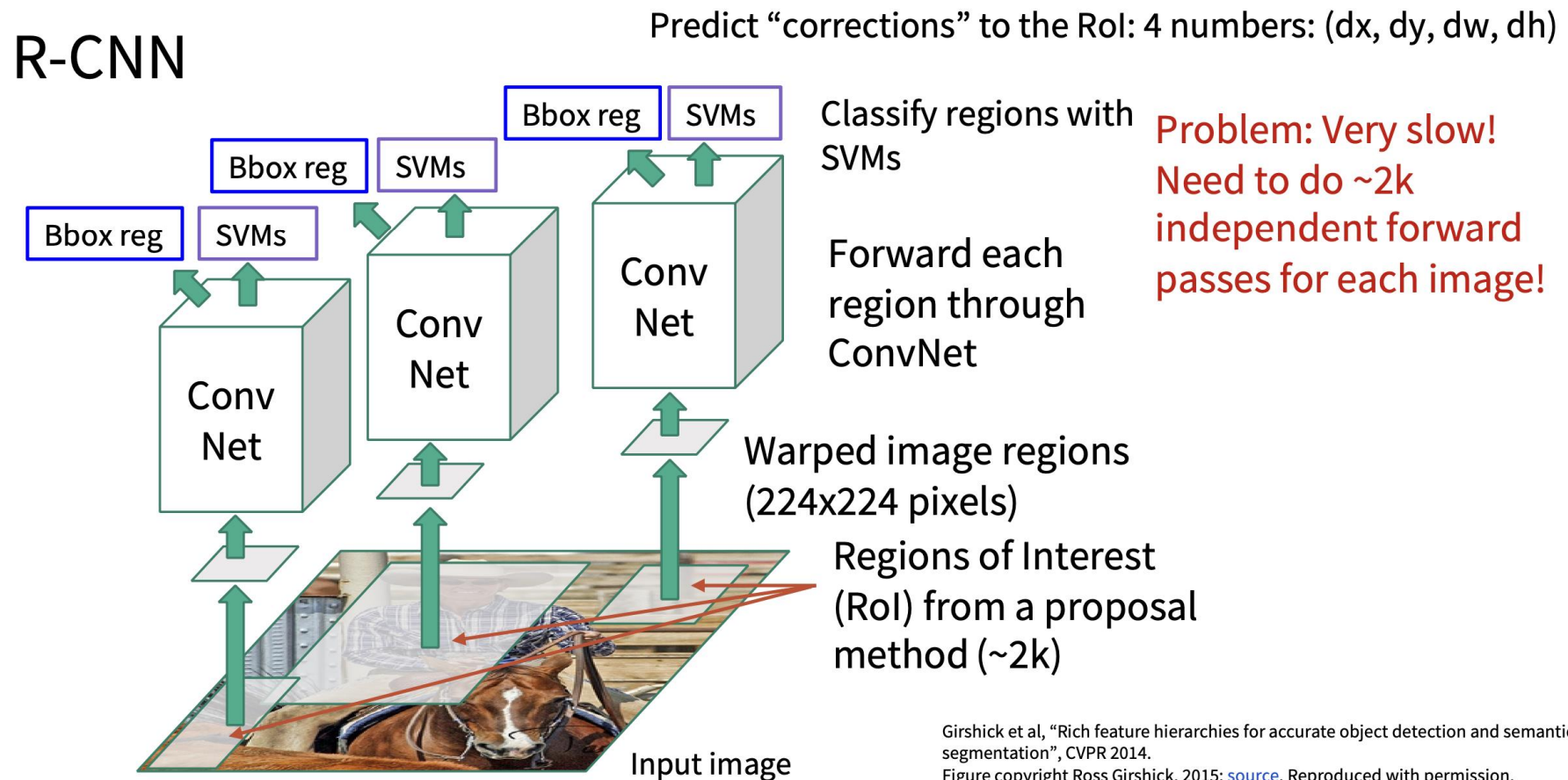
Defining the offsets for training:

$$\begin{aligned}d_x &= \frac{b_x - x_{proposal}}{w_{proposal}} & d_y &= \frac{b_y - y_{proposal}}{h_{proposal}} \\d_h &= \log\left(\frac{b_h}{h_{proposal}}\right) & d_w &= \log\left(\frac{b_w}{w_{proposal}}\right)\end{aligned}$$

$(x_{proposal}, y_{proposal}, w_{proposal}, h_{proposal})$ are the center coordinates, width, and height of the **region proposal**.

Object Detection

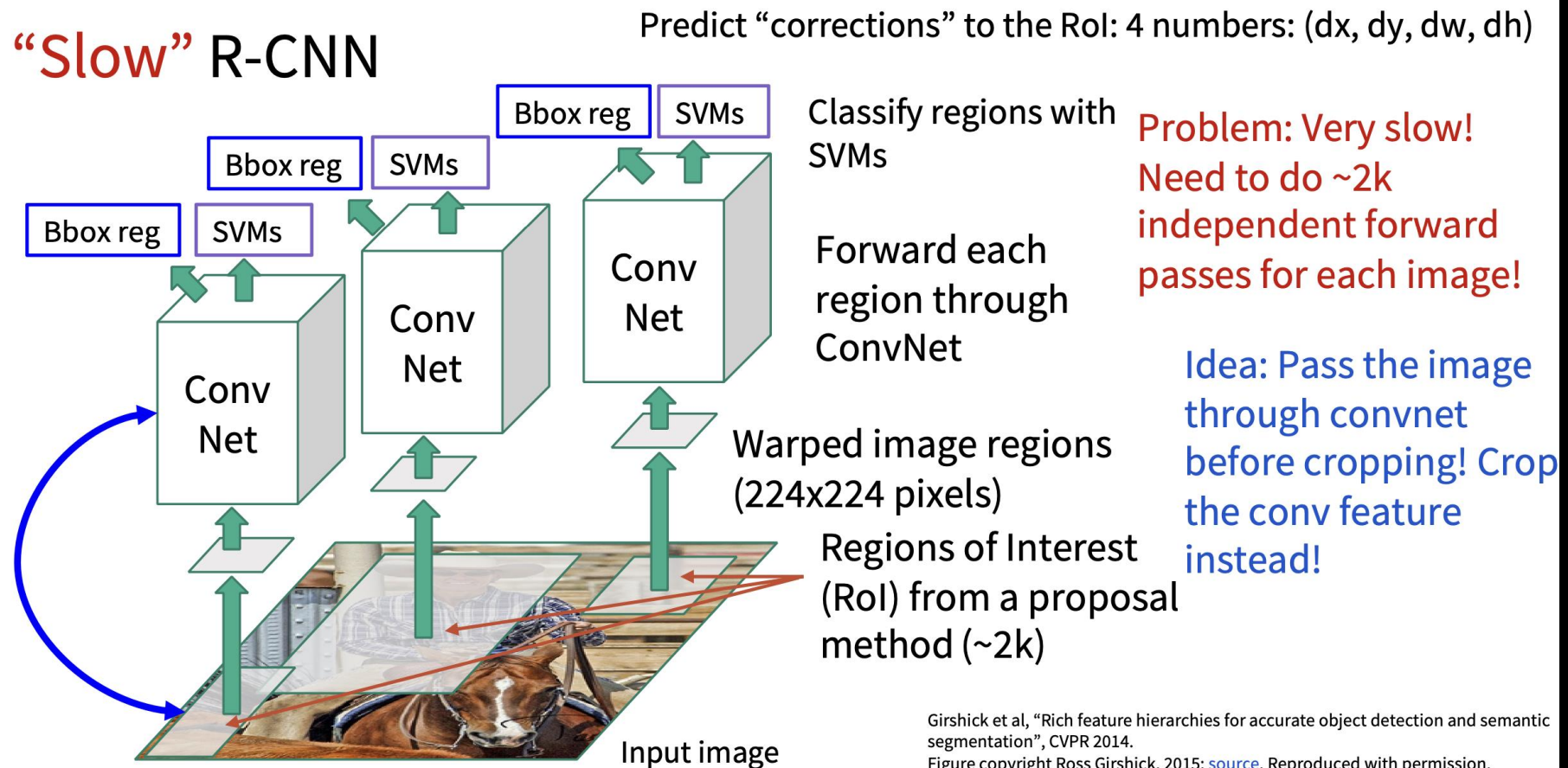
Regional CNN (R-CNN)



Girshick et al, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Object Detection

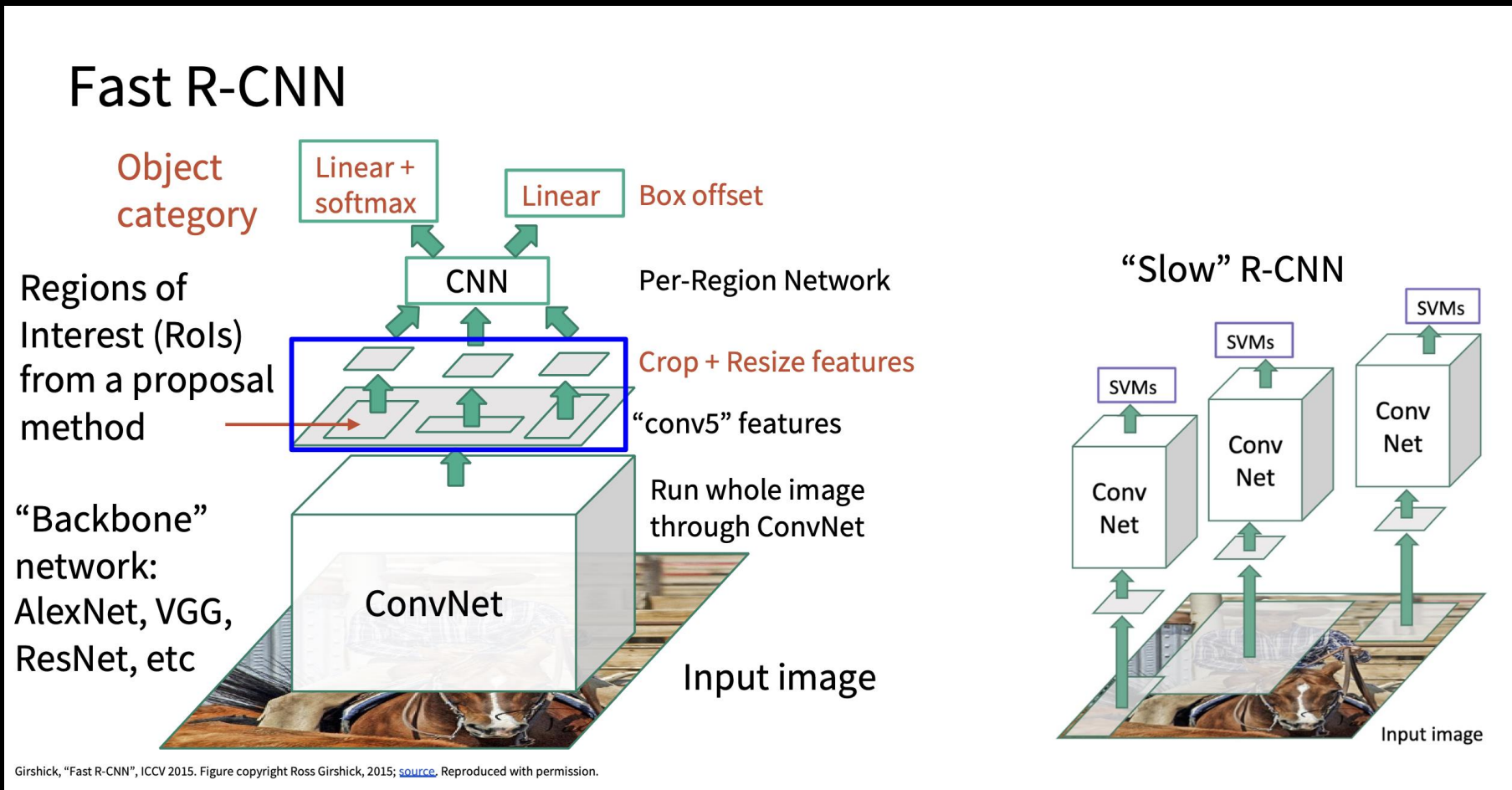
Regional CNN (R-CNN)



Girshick et al, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

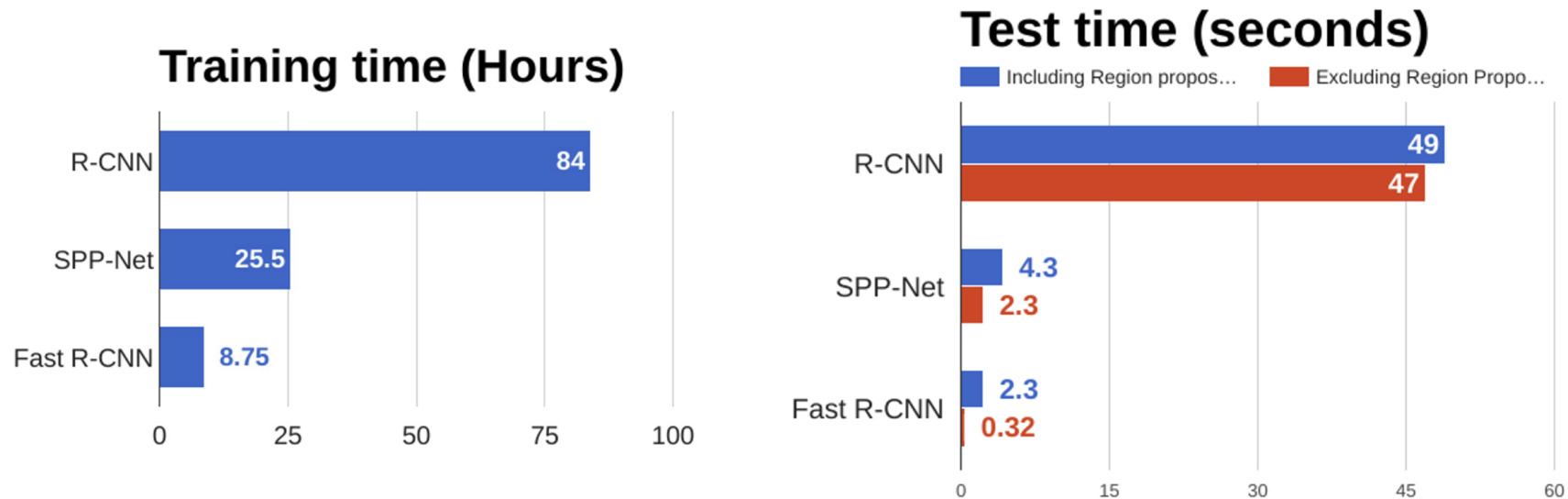
Object Detection

Faster R-CNN



Object Detection

R-CNN vs Fast R-CNN



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

He et al, "Spatial pyramid pooling in deep convolutional networks for visual recognition", ECCV 2014

Girshick, "Fast R-CNN", ICCV 2015



Object Detection

Open Source Frameworks

- Lots of good implementations on GitHub! TensorFlow Detection API:
- https://github.com/tensorflow/models/tree/master/research/object_detection
- Faster RCNN, SSD, RFCN, Mask R-CNN, ...
- Detectron2 (PyTorch)
- <https://github.com/facebookresearch/detectron2>
- Mask R-CNN, RetinaNet, Faster R-CNN, RPN, Fast R-CNN, R-FCN, ...
Finetune on your own dataset with pre-trained models

Term Project

➤ Term Project

- Make a team (4~6 students in a team)
- Choose a project topic (free topic, should be not the same with other teams), (Week 9)
- Make a project proposal (Week 10)
- Prepare a report of the project

➤ Report of the project

- Role of each team member and contribution
- Objective
- Definition
- Block diagrams of the application
- Demo video (Show your project while presentation)



